
Posterior Behavioral Cloning: Pretraining BC Policies for Efficient RL Finetuning

Andrew Wagenmaker¹ Perry Dong² Raymond Tsao¹ Chelsea Finn² Sergey Levine¹

Abstract

Standard practice across domains from robotics to language is to first pretrain a policy on a large-scale demonstration dataset, and then finetune this policy, typically with reinforcement learning (RL), in order to improve performance on deployment domains. This finetuning step has proved critical in achieving human or super-human performance, yet while much attention has been given to developing more effective finetuning algorithms, little attention has been given to ensuring the pretrained policy is an effective initialization for RL finetuning. In this work we seek to understand how the pretrained policy affects finetuning performance, and how to pretrain policies in order to ensure they are effective initializations for finetuning. We first show theoretically that standard behavioral cloning (BC) can fail to ensure coverage over the demonstrator’s actions, a minimal condition necessary for effective RL finetuning. We then show that if, instead of exactly fitting the observed demonstrations, we train a policy to model the *posterior* distribution of the demonstrator’s behavior given the demonstration dataset, we *do* obtain a policy that ensures coverage over the demonstrator’s actions, enabling more effective finetuning. Furthermore, this policy achieves this while ensuring pretrained performance is no worse than that of the BC policy. We then show this approach is practically implementable with modern generative models and leads to significantly improved RL finetuning performance on both realistic robotic control benchmarks and real-world robotic manipulation tasks, as compared to standard behavioral cloning.

¹Department of Electrical Engineering & Computer Sciences, UC Berkeley, Berkeley, CA, USA ²Department of Computer Science, Stanford University, Stanford, CA, USA. Correspondence to: Andrew Wagenmaker <ajwagen@berkeley.edu>.

1. Introduction

Across domains—from language, to vision, to robotics—a common paradigm has emerged for training highly effective “policies”: collect a large set of demonstrations, “pretrain” a policy via behavioral cloning (BC) to mimic these demonstrations, then “finetune” the pretrained policy on a deployment domain of interest. While pretraining can endow the policy with generally useful abilities, the finetuning step has proved critical in obtaining effective performance, enabling human value alignment and reasoning capabilities in language domains (Ouyang et al., 2022; Bai et al., 2022a; Team et al., 2025; Guo et al., 2025a), and improving task solving precision and generalization to unseen tasks in robotic domains (Nakamoto et al., 2024; Chen et al., 2025c; Kim et al., 2025; Wagenmaker et al., 2025a). In particular, reinforcement learning (RL)-based finetuning—where the pretrained policy is deployed in a setting of interest and its behavior updated based on the outcomes of these online rollouts—is especially crucial in improving the performance of a pretrained policy.

Critical to achieving successful RL-based finetuning performance in many domains—particularly in settings when policy deployment is costly and time-consuming, such as robotic control—is sample efficiency: effectively modifying the behavior of the pretrained model using as few deployment rollouts as possible. While significant attention has been given to developing more efficient finetuning algorithms, this ignores a primary ingredient in the RL finetuning process: the pretrained policy itself. Though generally more effective pretrained policies are the preferred initialization for finetuning (Guo et al., 2025a; Yue et al., 2025), it is not well understood how pretraining impacts finetuning performance beyond this, and how we might pretrain policies to enable more efficient RL finetuning.

In this work we seek to understand the role of the pretrained policy in RL finetuning, and how we might pretrain policies that (a) enable efficient RL finetuning, and (b) before finetuning, perform no worse than the policy pretrained with standard BC. We propose a novel pretraining approach—*posterior behavioral cloning* (POSTBC)—which, rather than fitting the empirical distribution of demonstrations as standard BC does, instead fits the *posterior*

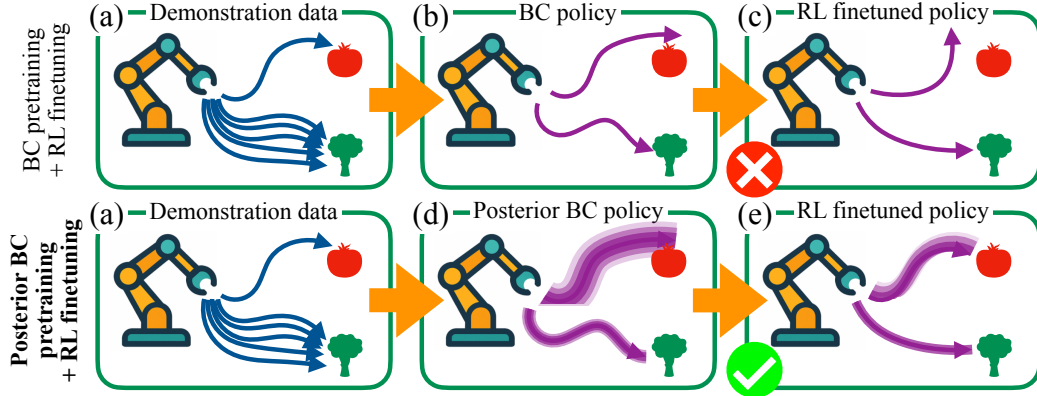


Figure 1. (a) We consider the setting where we are given demonstration data for some tasks of interest. (b) Standard BC pretraining fits the behaviors in the demonstrations, leading to effective performance in regions with high demonstration data density, yet can overcommit to the observed behaviors in regions with low data density. (c) This leads to ineffective RL finetuning, since rollouts from the BC policy provide little meaningful reward signal in such low data density regions, which is typically necessary to enable effective improvement. (d) In contrast, we propose *posterior behavioral cloning* (POSTBC), which instead of directly mimicking the demonstrations, trains a generative policy to fit the *posterior distribution* of the demonstrator’s behavior. This endows the pretrained policy with a wider distribution of actions in regions of low demonstrator data density, while in regions of high data density it reduces to approximately the standard BC policy. (e) This wider action distribution in low data density regions allows for collection of diverse observations with more informative reward signal, enabling more effective RL finetuning, while in regions of high data density performance converges to that of the demonstrator.

rior distribution over the demonstrator’s behavior. That is, assuming a uniform prior over the demonstrator’s behavior and viewing the demonstration data as samples from the demonstrator’s behavioral distribution, we seek to train a policy that models the posterior distribution of the demonstrator’s behavior given these observations. This enables the pretrained policy to take into account its potential uncertainty about the demonstrator’s behavior, and adjust the entropy of its action distribution based on this uncertainty. In states where it is uncertain about the demonstrator’s actions, POSTBC samples from a high-entropy distribution, allowing for a more diverse set of actions that may enable further policy improvement, while in states where it is certain about the demonstrator’s actions, it samples from a low-entropy distribution, simply mimicking what it knows to be the (correct) demonstrator behavior (see Figure 1).

We show that POSTBC leads to provable improvements over standard BC in terms of the potential for downstream RL performance. In particular, we focus on the ability of the pretrained policy to cover the demonstrator policy’s actions—whether it samples all actions the demonstrator policy might sample—which, for finetuning approaches that rely on rolling out the pretrained policy, is a prerequisite to ensure finetuning can match the performance of the demonstrator. We show standard BC can provably fail to cover the demonstrator’s distribution, while POSTBC *does* cover the demonstrator’s distribution, incurs no suboptimality in the performance of the pretrained policy as compared to the BC policy, and achieves a near-optimal sampling cost out of all policy estimators which have pretrained performance no worse than the BC policy’s.

Inspired by this, we develop a practical approach to approximating the posterior of the demonstrator in continuous action domains, and instantiate POSTBC with modern generative models—diffusion models—on robotic control tasks. Our instantiation relies only on pretraining with scalable supervised learning objectives—no RL is required in pretraining—and can be incorporated into existing BC training pipelines with minimal modification. We demonstrate experimentally that POSTBC pretraining can lead to significant performance gains in terms of the efficiency and effectiveness of RL finetuning, as compared to running RL finetuning on a policy pretrained with standard BC, and achieves these gains without decreasing the performance of the pretrained policy itself. We show that this holds for a variety of finetuning algorithms—both policy-gradient-style algorithms, and algorithms which explicitly refine or filter the distribution of the pretrained policy—enabling effective RL finetuning across a variety of challenging robotic tasks in both simulation and the real world.

2. Related Work

We highlight the most relevant work here. Please see Section A for discussion on other pretraining approaches (imitation learning, meta-learning, and reinforcement learning), methods for RL finetuning, and posterior sampling.

BC pretraining. BC training of expressive generative models—where the model is trained to predict the next “action” of the demonstrator—forms the backbone of pretraining for LLMs (Radford et al., 2018) and robotic control (Bojarski, 2016; Zhang et al., 2018; Rahmatizadeh et al., 2018; Stepputtis et al., 2020; Shafiullah et al., 2022; Gu

et al., 2023; Team et al., 2024; Zhao et al., 2024; Black et al., 2024; Kim et al., 2024). Experimentally, we focus in particular on policies parameterized as diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020), which have seen much attention in the robotics community (Chi et al., 2023; Ankile et al., 2024a; Zhao et al., 2024; Ze et al., 2024; Sridhar et al., 2024; Dasari et al., 2024; Team et al., 2024; Black et al., 2024; Bjorck et al., 2025), yet our approach can extend to other generative model classes as well. These works, however, simply pre-train with standard BC, and do not consider how the pre-training may affect RL finetuning performance.

Pretraining for downstream finetuning. Several recent works in the language domain aim to understand the relationship between pretraining and downstream finetuning (Springer et al., 2025; Zeng et al., 2025; Chen et al., 2025b; Jin et al., 2025; Chen et al., 2025a). A common thread through these works is that cross entropy loss is not predictive of downstream finetuning performance, and, in fact, low cross entropy loss can be anti-correlated with finetuning performance as the model can become *overconfident*. Most related to our work is the concurrent work of Chen et al. (2025a), which consider a notion of *coverage* closely related to our notion of *demonstrator action coverage*, and show that coverage generalizes faster than cross-entropy, and is an effective predictor of the downstream success of Best-of- N sampling. While both our work and Chen et al. (2025a) focus on notions of coverage to enable downstream improvement, we see this work as complementary. Chen et al. (2025a) show coverage generalizes faster than cross-entropy, yet our results show BC pretraining can *still* fail to ensure meaningful coverage. Furthermore, Chen et al. (2025a) does not consider the tradeoff between policy performance and coverage that is a primary focus of our work, and their proposed pretraining intervention—gradient normalization—would not resolve the shortcomings of BC in our setting. Finally, Chen et al. (2025a) is primarily a theoretical work and focuses on discrete next-token prediction (indeed, all works cited above consider only discrete next-token prediction); in contrast, a primary focus of our work is on continuous control, and we demonstrate our approach scales to real-world robotic settings.

3. Preliminaries

Mathematical notation. Let $\lesssim$ denote inequality up to absolute constants, $\Delta_{\mathcal{X}}$ the simplex over $\mathcal{X}$, and $\text{unif}(\mathcal{X})$ the uniform distribution over $\mathcal{X}$. $\mathbb{I}\{\cdot\}$ denotes the indicator function, $\mathbb{E}^\pi[\cdot]$ the expectation under policy π and $\mathbb{E}[\cdot]$ the expectation over the demonstrator dataset.

Markov decision processes. We consider decision-making in the context of episodic, fixed-horizon Markov decision processes (MDPs). An MDP $\mathcal{M}$ is denoted by

a tuple $(\mathcal{S}, \mathcal{A}, \{P_h\}_{h=1}^H, P_0, r, H)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ the set of actions, $P_h : \mathcal{S} \times \mathcal{A} \rightarrow \Delta_{\mathcal{S}}$ the next-state distribution at step h , $P_0 \in \Delta_{\mathcal{S}}$ the initial state distribution, $r_h : \mathcal{S} \times \mathcal{A} \rightarrow \Delta_{[0,1]}$ the reward distribution, and H the horizon. Interaction with $\mathcal{M}$ proceeds in episodes of length H . At step 1, we sample a state $s_1 \sim P_0$, take an action $a_1 \in \mathcal{A}$, receive reward $r_1(s_1, a_1)$, and transition to state $s_2 \sim P_1(\cdot | s_1, a_1)$. This continues for H steps until the MDP resets. We let $\mathcal{J}(\pi) := \mathbb{E}^\pi[\sum_{h=1}^H r_h(s_h, a_h)]$ denote the expected reward for policy π . In general, our goal is to find a policy that maximizes $\mathcal{J}(\pi)$.

Behavioral cloning. We assume we are given some dataset $\mathcal{D} = \{(s_1^t, a_1^t, \dots, s_H^t, a_H^t)\}_{t=1}^T$ collected by running a *demonstrator* policy π^β on $\mathcal{M}$, so that $(s_1^t, a_1^t, \dots, s_H^t, a_H^t)$ denotes a trajectory rollout of π^β , with $a_h^t \sim \pi_h^\beta(\cdot | s_h^t)$. We assume π^β is Markovian but otherwise make no further assumptions on it (so, in particular, π^β may be stochastic and suboptimal). Our demonstrator dataset does not include reward labels—preventing standard offline RL approaches from applying—but we assume access to reward labels during online interactions.

Behavioral cloning (BC) attempts to fit a policy $\hat{\pi}^{\text{bc}}$ to match the distribution of actions observed in $\mathcal{D}$. Typically this is achieved via supervised learning, where $\hat{\pi}^{\text{bc}}$ is trained to predict a given s for $(s, a) \in \mathcal{D}$. In the tabular setting, which we consider in Section 4, the natural choice for $\hat{\pi}^{\text{bc}}$ models the empirical action distribution in $\mathcal{D}$:

$$\hat{\pi}_h^{\text{bc}}(a | s) := \frac{T_h(s, a)}{T_h(s)} \cdot \mathbb{I}\{T_h(s) > 0\} + \text{unif}(\mathcal{A}) \cdot \mathbb{I}\{T_h(s) = 0\} \quad (1)$$

where $T_h(s, a) = \sum_{t=1}^T \mathbb{I}\{(s_h^t, a_h^t) = (s, a)\}$ and $T_h(s) = \sum_{t=1}^T \mathbb{I}\{s_h^t = s\}$. The following result bounds the suboptimality of this estimator, and shows that it is optimal.

Proposition 1 (Rajaraman et al. (2020)). *If $\mathcal{D}$ contains T demonstrator trajectories, we have $\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}})] \lesssim \frac{H^2 S \log T}{T}$. Furthermore, for any estimator $\hat{\pi}$, there exists some MDP $\mathcal{M}$ and demonstrator π^β such that $\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi})] \gtrsim \min\{H, \frac{H^2 S}{T}\}$.*

In other words, without additional reward information, we cannot in general hope to obtain a policy from $\mathcal{D}$ that does better than (1), in terms of the reward collected by the pre-trained policy. Note that the estimator (1) is, under the uniform demonstrator prior (i.e. the demonstrator is equally likely to play each action in each state), the *maximum a posteriori* (MAP) estimate of the demonstrator’s behavior.

4. Achieving Demonstrator Action Coverage via Posterior Sampling

In this section we seek to understand how pretraining affects the ability to further improve the downstream policy

with RL finetuning, and how we might pretrain to enable downstream improvement. For simplicity, in this section we assume $\mathcal{M}$ is tabular, and let S and A denote the cardinalities of the state and action spaces, respectively.

4.1. Demonstrator Action Coverage

The performance of RL finetuning depends significantly on the RL algorithm applied. Rather than limiting our results to a particular RL algorithm, we instead focus on what is often a prerequisite for effective application of any such approach—demonstrating that the *support* of the pretrained policy is sufficient to enable improvement. In particular, we consider the following definition for the “effective” support of a policy, relative to the demonstrator policy π^β .

Definition 4.1 (Demonstrator Action Coverage). We say policy π achieves demonstrator action coverage with parameter $\gamma > 0$ if, for all $(s, h) \in \mathcal{S} \times [H]$ and $a \in \mathcal{A}$, we have $\pi_h(a | s) \geq \gamma \cdot \pi_h^\beta(a | s)$.

The majority of RL finetuning approaches rely on rolling out the pretrained policy—which we denote as $\hat{\pi}^{\text{pt}}$ —online, and using the collected observations to finetune its behavior. If our pretrained policy achieves demonstrator action coverage with parameter γ , then this ensures that any action sampled by π^β will also be sampled by $\hat{\pi}^{\text{pt}}$ in these rollouts (with some probability). While this is not a *sufficient* condition for online improvement, it is a *necessary* condition for performing as well as the demonstrator π^β (as Proposition 2 shows), and is therefore also a necessary condition for improving over π^β . Furthermore, the *value* of γ has impact on the cost of RL finetuning. A policy π which achieves demonstrator action coverage with parameter γ requires a factor of $1/\gamma$ more samples than π^β to ensure it samples some action in the support of π^β . For approaches such as Best-of- N sampling that rely on sampling many actions from the pretrained policy and then taking the best one, a large value of γ ensures we can efficiently sample actions likely to be sampled by the demonstrator policy π^β , while if γ is small, it may take a significant number of samples to sample an action necessary for improvement.

Problem Statement: Demonstrator Action Coverage with BC-Pretrained Performance. In the following, we aim to understand how we can pretrain policies that (a) achieve demonstrator action coverage with values of γ as large as possible and (b) have the same pretrained performance as the BC-pretrained performance.

4.2. BC Fails to Achieve Demonstrator Action Coverage

We first consider standard BC, i.e. (1). The following result shows that the estimator in (1), despite achieving the best possible suboptimality rate, can fail to achieve a meaningful guarantee on demonstrator action coverage.

Proposition 2. Fix $\epsilon \in (0, 1/8]$. Then there exist some MDPs $\mathcal{M}^1, \mathcal{M}^2$ and demonstrator policy π^β such that, if $\mathcal{M} \in \{\mathcal{M}^1, \mathcal{M}^2\}$, unless $T \geq \frac{1}{20\epsilon}$, we have that, with probability at least $1/2$: $\mathcal{J}(\pi^\beta) - \epsilon > \max_{\pi \in \hat{\Pi}} \mathcal{J}(\pi)$, for $\hat{\Pi} := \{\pi : \pi_h(a | s) = 0 \text{ if } \hat{\pi}_h^{\text{bc}}(a | s) = 0, \forall s, a, h\}$. Furthermore, for any $T' > 0$,

$$\min_{\hat{\pi}^{T'}} \max_{i \in \{1, 2\}} \mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}} [\max_{\pi} \mathcal{J}^{\mathcal{M}^i}(\pi) - \mathcal{J}^{\mathcal{M}^i}(\hat{\pi}^{T'})] \geq \frac{1}{2},$$

where $\mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}}[\cdot]$ denotes the expectation over trajectories generated by rolling out $\hat{\pi}^{\text{bc}}$ on $\mathcal{M}^i$, and $\hat{\pi}^{T'}$ is a policy estimator obtained after T' such rollouts.

Proposition 2 shows that, unless we have a sufficiently large demonstration dataset, half of the time (i.e. half of the random draws of the demonstrator dataset) the policy returned by standard BC will not contain a near-optimal policy in its support and, furthermore, rolling out $\hat{\pi}^{\text{bc}}$ on $\mathcal{M}$ will not allow us to learn a near-optimal policy. This also shows that demonstrator action coverage is, in some cases, a necessary condition for RL improvement—without it, we will not sample actions played by the demonstrator, and will be unable to determine which lead to the best performance.

A straightforward solution to this shortcoming of BC is to simply add exploration noise to our pretrained policy—rather than playing $\hat{\pi}^{\text{bc}}$ at every step, with some probability play a random action. While this will clearly ensure demonstrator action coverage is achieved, the following result shows there is a fundamental tradeoff between policy suboptimality and the value of γ .

Proposition 3. Fix $T > 0$, $H \geq 2$, $S \geq \lceil \log_2 4T \rceil + 2$, $\xi \geq 0$, define $\epsilon := \frac{H^2 S \log T}{T} + \xi$, and assume $\epsilon \leq \frac{1}{2}$. Define the policy $\hat{\pi}^{\text{u}, \alpha}$ as $\hat{\pi}_h^{\text{u}, \alpha}(\cdot | s) := (1 - \alpha) \cdot \hat{\pi}_h^{\text{bc}}(\cdot | s) + \alpha \cdot \text{unif}(\mathcal{A})$. Then there exists some MDP $\mathcal{M}$ with S states, 2 actions, and horizon H where, in order to ensure:

1. $\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{u}, \alpha})] \leq \epsilon$,
2. $\hat{\pi}^{\text{u}, \alpha}$ achieves demonstrator action coverage with parameter γ and probability $1 - \delta$, for $\delta \in (0, 1/4e)$,

we must have $\alpha \leq 32\epsilon$ and $\gamma \leq \frac{64}{A} \cdot \epsilon$. Furthermore, with probability at least $1/4e$, $\mathcal{J}(\pi^\beta) - \frac{1}{T} \cdot \epsilon > \max_{\pi \in \hat{\Pi}} \mathcal{J}(\pi)$.

In order to achieve the $\frac{H^2 S \log T}{T}$ suboptimality rate achieved by standard BC, Proposition 3 shows that we can only guarantee demonstrator action coverage with $\gamma \lesssim \frac{1}{A} \cdot \frac{H^2 S \log T}{T}$. In other words, to ensure we sample a particular action from $\hat{\pi}^{\text{u}, \alpha}$ that is sampled by π^β , it will require sampling a factor of $\frac{AT}{H^2 S \log T}$ more samples from $\hat{\pi}^{\text{u}, \alpha}$ than it would require from π^β , which could be a significant cost when T is large. Furthermore, Proposition 3 shows that this limitation is critical—if we seek to shortcut this exploration and set $\alpha \leftarrow 0$, we will fail to match the performance of π^β completely.

4.3. Posterior Demonstrator Policy Achieves

Demonstrator Action Coverage

Next we show that a mixture of the standard BC policy and the *posterior* on the demonstrator’s policy achieves a near optimal balance between policy suboptimality and demonstrator action coverage.

Definition 4.2 (Posterior Demonstrator Policy). Given prior distribution $P_{\text{prior}}^\beta \in \Delta_\Pi$ over demonstrator policies, let $P_{\text{post}}^\beta(\cdot \mid \mathcal{D})$ denote the posterior distribution given demonstration dataset $\mathcal{D}$. We then define the *posterior demonstrator policy* $\hat{\pi}^{\text{post}}$ as $\hat{\pi}_h^{\text{post}}(a \mid s) := \mathbb{E}_{\pi \sim P_{\text{post}}^\beta(\cdot \mid \mathcal{D})}[\pi_h(a \mid s)]$.

$\hat{\pi}^{\text{post}}$ is therefore the expected posterior policy of the demonstrator under prior P_{prior}^β given observations $\mathcal{D}$. Critically, this takes into account the entire posterior distribution of the demonstrator’s behavior, in contrast to the MAP estimate produced by standard BC, which simply returns a point estimate of the behavior. In practice, we require a slightly regularized version of $\hat{\pi}^{\text{post}}$, $\hat{\pi}^{\text{post},\lambda}$, which is identical to $\hat{\pi}^{\text{post}}$ if $HT \lesssim e^A$, and otherwise adds a small amount of additional regularization (see Section B.3 for a precise definition). We have the following.

Theorem 1. Let P_{prior}^β be the uniform distribution over Markovian policies, and set $\hat{\pi}^{\text{pt}}$ to

$$\hat{\pi}_h^{\text{pt}}(a \mid s) = (1 - \alpha) \cdot \hat{\pi}_h^{\text{bc}}(a \mid s) + \alpha \cdot \hat{\pi}_h^{\text{post},\lambda}(a \mid s) \quad (2)$$

for $\alpha = \frac{1}{\max\{A, H, \log(HT)\}}$. Then

$$\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{pt}})] \lesssim \frac{H^2 S \log T}{T},$$

and with probability at least $1 - \delta$, for all (s, a, h) ,

$$\hat{\pi}_h^{\text{pt}}(a \mid s) \gtrsim \frac{1}{A + H + \log(HT)} \cdot \min \left\{ \frac{\pi_h^\beta(a \mid s)}{\log(SH/\delta)}, \frac{1}{A + \log(HT)} \right\}.$$

Theorem 1 shows that by setting $\hat{\pi}^{\text{pt}}$ to a mixture of the BC policy and the posterior demonstrator policy, we obtain the same suboptimality guarantee as standard BC. Furthermore, this policy achieves demonstrator action coverage with $\gamma \approx 1/(A + H)$, only requiring a factor of $\approx A + H$ more samples to ensure we sample a particular action from π^β than π^β . We refer to this approach as *posterior behavioral cloning* (POSTBC). The following result shows that the scaling in γ POSTBC achieves is nearly unimprovable.

Theorem 2. Fix any $A > 1$ and $T > 1$. Then there exists a family of MDPs $\{\mathcal{M}^i\}_{i \in [A]}$ such that each $\mathcal{M}^i$ has A actions and $S = H = 1$, and if any estimator $\hat{\pi}$ satisfies $\mathcal{J}^{\mathcal{M}^i}(\pi^{\beta,i}) - \mathbb{E}^{\mathcal{M}^i}[\mathcal{J}(\hat{\pi})] \leq c \cdot \frac{H^2 S \log T}{T}$ for all $i \in [A]$ and some constant $c > 0$, then for $\hat{\pi}$ to achieve demonstrator action coverage with respect to $\pi^{\beta,i}$ on each $\mathcal{M}^i$ with probability at least $\delta \in (0, 1/4]$, we must have $\gamma \leq c \cdot \frac{\log T}{A}$.

Theorem 2 shows that, to match the suboptimality guarantee of the BC policy, no estimator can achieve demonstrator action coverage with γ larger than $\approx 1/A$. Thus, the demonstrator action coverage achieved by POSTBC is nearly unimprovable, matching the lower bound as long as $H \leq A$. In other words, if we want a policy that preserves the optimality of $\hat{\pi}^{\text{bc}}$ while playing a diverse enough action distribution to enable further online improvement, mixing the posterior demonstrator policy with the BC policy achieves a near-optimal tradeoff. This is in contrast to the BC policy, which does not achieve a meaningful guarantee on demonstrator action coverage, as well as the BC policy augmented with random exploration, which in order to match the suboptimality of the BC policy achieves a very suboptimal guarantee on demonstrator action coverage.

5. Practical Posterior Behavioral Cloning

Next, we show how POSTBC can be instantiated in continuous control settings using expressive generative models.

5.1. Sampling from the Posterior Demonstrator Policy for Gaussian Demonstrators

To motivate our practical instantiation, assume:

$$\pi_h^\beta(\cdot \mid s) = \mathcal{N}(\mu_h(s), \sigma_h^2(s) \cdot I),$$

for some (unknown) $\mu_h(s) \in \mathbb{R}^d$ and (known) $\sigma_h(s) \in \mathbb{R}$. Assume we have observations $\mathcal{D} = \{a_1, \dots, a_T\} \sim \pi_h^\beta(\cdot \mid s)$, and a $\mathcal{N}(0, I)$ prior on $\mu_h(s)$. Our theory suggests that instead of fitting the BC policy, we should fit the posterior demonstrator policy $\hat{\pi}^{\text{post}}$. In this Gaussian setting, it is straightforward to show that $\hat{\pi}_h^{\text{post}}(\cdot \mid s)$ is the distribution:

$$\mathcal{N}\left(\frac{1}{\sigma_h^2(s) + T} \cdot \sum_{t=1}^T a_t, \frac{\sigma_h^2(s)}{\sigma_h^2(s) + T} \cdot I + \sigma_h^2(s) \cdot I\right).$$

While in the Gaussian setting we can easily sample from this distribution, we wish to motivate a generalizable procedure that extends to settings where sampling is less straightforward. To this end, we first note that the BC policy (the MAP estimator) is simply the distribution $\mathcal{N}(\frac{1}{\sigma_h^2(s) + T} \cdot \sum_{t=1}^T a_t, \sigma_h^2(s) \cdot I)$. To generate a sample from $\hat{\pi}_h^{\text{post}}(\cdot \mid s)$, it then suffices to sample from the BC policy and perturb the sample by noise $w \sim \mathcal{N}(0, \frac{\sigma_h^2(s)}{\sigma_h^2(s) + T} \cdot I)$. The following result, an extension of Osband et al. (2018), shows that there is a close connection between this noise distribution and the posterior on $\mu_h(s)$, and that we can generate a sample from the posterior on $\mu_h(s)$ with a simple optimization procedure.

Proposition 4. We have $P_{\text{post}}^\beta(\cdot \mid \mathcal{D}) = \mathcal{N}(\frac{1}{\sigma_h^2(s) + T} \cdot \sum_{t=1}^T a_t, \frac{\sigma_h^2(s)}{\sigma_h^2(s) + T} \cdot I)$ and, if we set

$$\hat{\mu}_h(s) = \arg \min_{\mu} \sum_{i=1}^T \|\mu - \tilde{a}_i\|_2^2 + \sigma_h^2(s) \cdot \|\mu - \tilde{\mu}_h(s)\|_2^2,$$

for $\tilde{a}_t = a_t + w_t$, $w_t \sim \mathcal{N}(0, \sigma_h^2(s) \cdot I)$, and $\tilde{\mu}_h(s) \sim \mathcal{N}(0, I)$, then $\hat{\mu}_h(s) \sim P_{\text{post}}^\beta(\cdot \mid \mathcal{D})$.

Thus, to generate a sample $w \sim \mathcal{N}(0, \frac{\sigma_h^2(s)}{\sigma_h^2(s)+T} \cdot I)$, we can first generate samples $\hat{\mu}_h(s)$, compute their empirical variance, $\text{cov}_h(s)$, and sample from a Gaussian with mean 0 and variance $\text{cov}_h(s)$. Altogether, we have the following procedure to sample $\hat{a} \sim \hat{\pi}_h^{\text{post}}(\cdot \mid s)$:

1. Compute the BC policy $\hat{\pi}^{\text{bc}}$ from observations $\mathcal{D}$.
2. Compute samples from the posterior P_{post}^β using the optimization procedure of Proposition 4 and estimate the variance $\text{cov}_h(s)$ of the posterior from these samples.
3. Generate samples $\tilde{a} \sim \hat{\pi}_h^{\text{bc}}(\cdot \mid s)$ and $w \sim \mathcal{N}(0, \text{cov}_h(s))$, and set $\hat{a} \leftarrow \tilde{a} + w$.

While in the Gaussian setting simpler methods would suffice to sample from $\hat{\pi}^{\text{post}}$, each step in this procedure can be easily extended to more complex settings, suggesting a generalizable approach to approximate samples from $\hat{\pi}^{\text{post}}$.

5.2. Practical Instantiation of POSTBC

In practice our data likely does not satisfy this Gaussianity assumption, and we also wish to incorporate function approximation. To accommodate this, we seek to approximate the procedure outlined above in more general settings, and summarize our approach in Algorithm 1.

Algorithm 1 Posterior Behavioral Cloning (POSTBC)

- 1: **input:** demonstration dataset $\mathcal{D}$, generative model class $\hat{\pi}^\theta$, ensemble size K , posterior weight α
- 2: Fit K predictors f_ℓ to perturbed $\mathcal{D}$ (see Algorithm 2)
- 3: Set $\text{cov}(\cdot)$ to covariance of $\{f_\ell\}_{\ell \in [K]}$
- 4: **for** $i = 1, 2, 3, \dots$ **do**
- 5: Sample batch $\mathcal{B}_i \sim \text{unif}(\mathcal{D})$
- 6: For all $(s, a) \in \mathcal{B}_i$, sample $w_s \sim \mathcal{N}(0, \text{cov}(s))$, and set $\tilde{\mathcal{B}}_i \leftarrow \{(s, a + \alpha \cdot w_s) : s \in \mathcal{B}_i\}$
- 7: Take gradient step on $\hat{\pi}^\theta$ for loss computed on $\tilde{\mathcal{B}}_i$
- 8: **return** $\hat{\pi}^\theta$

Algorithm 1 first trains an ensemble of predictors $\{f_\ell\}_{\ell \in [K]}$ on perturbed versions of $\mathcal{D}$, which it then uses to approximate the posterior covariance as in step 2 above. It then trains a generative policy to fit the demonstrations in $\mathcal{D}$, perturbed by noise of covariance $\text{cov}(\cdot)$, approximating step 3 above. This produces a single policy that approximates the posterior mixture in Equation (2), modeling the posterior demonstrator policy and, Theorem 1 suggests, leading to a more effective initialization for RL finetuning, instantiating the behavior illustrated in Figure 1.

Implementing Algorithm 1 only requires minor modification to standard generative policy training (simply train an

initial ensemble of predictors, then add noise to the action target for each batch sampled), and in particular does not require RL in pretraining. While any expressive generative model class can be used for $\hat{\pi}^\theta$, for all the following experiments we utilize a diffusion model. Please see Section D for further details on the practical instantiation of POSTBC.

6. Experiments

Finally, we seek to demonstrate that in practice POSTBC (a) enables more efficient RL finetuning of pretrained policies, and (b) produces a pretrained policy that itself performs effectively, on par with the BC pretrained policy. We focus on continuous control domains, in particular robotic control. We test on both the Robomimic (Mandlekar et al., 2021) and Libero (Liu et al., 2023) simulators, as well as a real-world WidowX 250 6-DoF robot arm. For Robomimic, we consider pretraining single-task policies on the provided set of demonstrations, in particular for the Lift, Can, and Square tasks. Libero contains a diverse set of robot manipulation tasks, and we focus in particular on the 16 tasks from Kitchen Scene 1–3 from the Libero 90 suite. We pretrain a single policy on the provided demonstrations across these tasks, then use this policy as the starting point for RL finetuning, and run single-task RL on the tasks from Kitchen Scene 1–3. In all cases, we instantiate $\hat{\pi}^{\text{pt}}$ with a diffusion model, a standard parameterization for BC policies in robotic control (Chi et al., 2023; Dasari et al., 2024; Team et al., 2024). In all cases, we use a binary success reward for RL finetuning. See Figures 5 and 6 for visualizations of our settings.

In principle, POSTBC can be combined with any RL finetuning algorithm, and we seek to demonstrate that it improves performance on a representative set of approaches. In particular, we consider DSRL (Wagenmaker et al., 2025a), which refines a pretrained diffusion policy’s

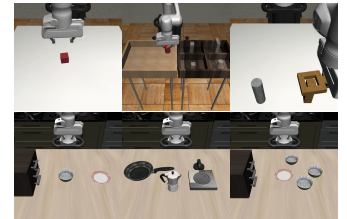


Figure 5. Robomimic and Libero settings.

distribution by running RL over its latent-noise space, DPPO (Ren et al., 2024), an on-policy policy-gradient-style algorithm for finetuning diffusion policies, and Best-of- N sampling. Best-of- N can be instantiated in a variety of ways (see e.g. Chen et al. (2022); Hansen-Estruch et al. (2023); He et al. (2024); Nakamoto et al. (2024); Dong et al. (2025b)). Here we instantiate it by rolling out the pretrained policy on the task of interest T_{on} times (where T_{on} is specified in our results) to collect trajectories labeled with reward, and train a Q -function via IQL (Kostrikov et al., 2021) on these trajectories. At test time, we again roll out the pretrained policy but at each state sample N

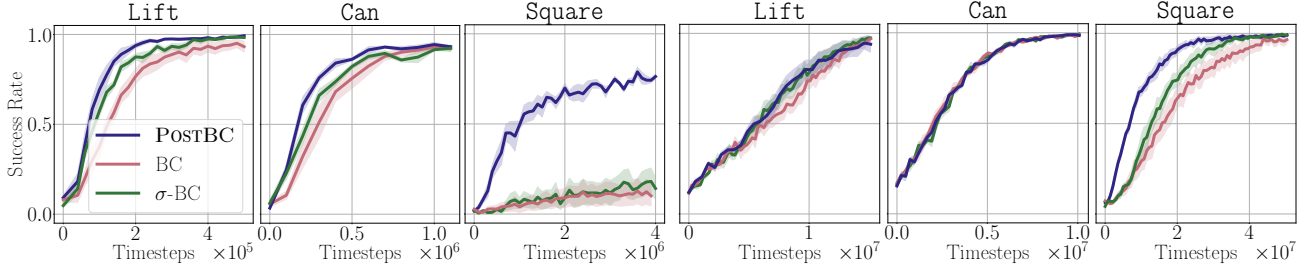


Figure 2. Comparison of DSRL finetuning performance combined with different BC pretraining approaches on Robomimic.

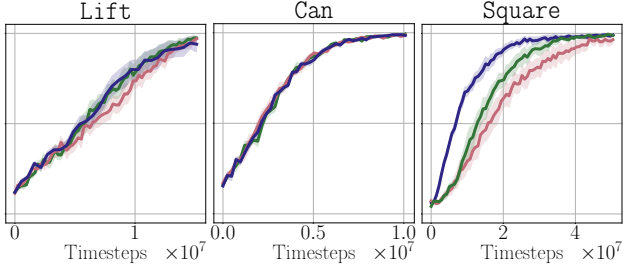


Figure 3. Comparison of DPPO finetuning performance combined with different BC pretraining approaches on Robomimic.

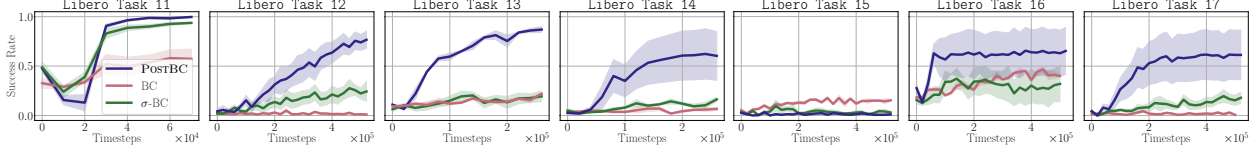


Figure 4. Comparison of DSRL finetuning performance on all tasks from Libero 90, Kitchen Scene 2.

actions from the policy, and play the action that has the largest value under this Q -function.

As baselines, we consider running standard BC pretraining on $\mathcal{D}$, as well as what we refer to as σ -BC, where instead of perturbing the actions in $\mathcal{D}$ by the posterior variance as in Algorithm 1, we instead perturb them by uniform, state-independent noise with variance σ^2 . This is then equivalent to POSTBC, except we set $\text{cov}(s) = \sigma^2 \cdot I$ for some fixed $\sigma > 0$ in Algorithm 1 (note that this is a continuous analog to the approach considered in Proposition 3). This itself is a novel approach and our theory predicts it too may lead to improved performance over pretraining with standard BC. On Robomimic, we also compare against VALUEDICE (Kostrikov et al., 2019) (which we abbreviate as DICE), as a representative non-BC imitation learning approach. Rather than training to match the demonstrator’s actions via supervised learning, VALUEDICE attempts to learn a policy with state distribution matching the state distribution of the demonstrations, and only requires access to offline demonstration data. All Robomimic results are averaged over 5 seeds, and Libero results are averaged over 3 seeds. For all experiments, error bars denote 1 standard error. See Section D for additional experimental details.

6.1. POSTBC Enables Efficient RL Finetuning

Our results from running DSRL on Robomimic are given in Figure 2 and on Libero in Figure 4. On Robomimic, POSTBC significantly outperforms both baselines on Square, and on Lift and Can requires roughly $2\times$ fewer samples to achieve 75% success than BC. For Libero, we run DSRL on all tasks from Kitchen Scene 2. We see that POSTBC pretraining leads to significant gains for Libero, enabling efficient RL finetuning in settings where both standard BC pretraining and σ -BC pretraining fail. In particular, on each task other

than Task 15, POSTBC leads to large gains—we hypothesize that on Task 15, where all methods fail to learn, this is due primarily to the poor pretrained policy performance for all methods, which often makes RL improvement challenging. Our Best-of- N results are given in Table 1. Across settings, POSTBC-pretraining leads to consistent improvements over both BC- and σ -BC-pretrained policies for Best-of- N , and also consistently outperforms VALUEDICE.

Our results for DPPO are given in Figure 3 where we see that POSTBC pretraining again leads to substantial gains on Square (approximately $2\times$ fewer samples to reach 75% performance compared to BC). This illustrates that POSTBC still improves performance even for RL finetuning algorithms that modify the weights of the pretrained policy, and for which the resulting actions are therefore not explicitly constrained to the actions played by the pretrained policy. We hypothesize that this is due to the ability of POSTBC to adapt its exploration based on its certainty of demonstrator behavior. POSTBC naturally increases exploration in states where it is uncertain what the demonstrator will do—states where exploration is required—while ensuring it does not over-explore in states where it knows that the demonstrator will do and exploration is not required. This is true whether we play an algorithm that modifies the weights of the pretrained policy or not, so we would expect improvement even when running with DPPO. Together, these results illustrate that POSTBC enables more effective RL finetuning on both single-task (Robomimic) and multi-task (Libero) settings, and across state- and image-based observations.

6.2. POSTBC Preserves Pretrained Performance

We next consider POSTBC pretrained performance. We provide results on pretrained performance for Robomimic

Task	Best-of- N (1000 Rollouts)				Best-of- N (2000 Rollouts)				Pretrained Performance	
	BC	σ -BC	DICE	POSTBC	BC	σ -BC	DICE	POSTBC	BC	POSTBC
Robomimic Lift	59.4 $\pm$ 2.2	61.5 $\pm$ 3.9	44.4 $\pm$ 7.7	74.2 $\pm$ 3.0	68.1 $\pm$ 2.2	76.1 $\pm$ 3.5	47.6 $\pm$ 7.8	81.3 $\pm$ 5.7	71.0 $\pm$ 0.5	69.7 $\pm$ 1.5
Robomimic Can	70.3 $\pm$ 1.7	77.9 $\pm$ 1.2	17.0 $\pm$ 5.7	75.0 $\pm$ 2.5	76.9 $\pm$ 1.5	82.4 $\pm$ 1.6	41.8 $\pm$ 8.4	84.5 $\pm$ 1.3	43.1 $\pm$ 0.9	44.7 $\pm$ 1.0
Robomimic Square	44.8 $\pm$ 0.7	48.1 $\pm$ 2.2	6.9 $\pm$ 0.9	52.4 $\pm$ 1.9	54.4 $\pm$ 1.3	54.2 $\pm$ 3.7	8.3 $\pm$ 1.3	56.8 $\pm$ 3.2	17.9 $\pm$ 0.7	18.1 $\pm$ 0.8
Libero Scene 1 (5 tasks)	37.7 $\pm$ 2.5	57.9 $\pm$ 3.4	-	67.0 $\pm$ 5.6	46.1 $\pm$ 2.6	63.1 $\pm$ 4.1	-	77.7 $\pm$ 1.3	23.6 $\pm$ 1.7	25.3 $\pm$ 1.2
Libero Scene 2 (7 tasks)	21.5 $\pm$ 1.2	26.9 $\pm$ 1.0	-	42.0 $\pm$ 2.2	23.9 $\pm$ 0.7	29.0 $\pm$ 1.6	-	49.5 $\pm$ 2.9	11.4 $\pm$ 0.2	12.3 $\pm$ 1.5
Libero Scene 3 (4 tasks)	47.7 $\pm$ 0.7	53.2 $\pm$ 2.3	-	63.3 $\pm$ 5.1	45.8 $\pm$ 3.3	60.8 $\pm$ 1.9	-	70.0 $\pm$ 0.4	39.5 $\pm$ 1.5	40.8 $\pm$ 1.3
Libero All (16 tasks)	33.1 $\pm$ 0.5	43.1 $\pm$ 0.8	-	55.1 $\pm$ 2.1	36.3 $\pm$ 0.3	47.6 $\pm$ 2.8	-	63.4 $\pm$ 1.3	22.2 $\pm$ 0.3	23.5 $\pm$ 0.5

Table 1. Comparison of success rates of Best-of- N sampling on Robomimic and Libero, for different pretraining approaches. Bolded text denotes best approach.

Table 2. Pretrained success on Robomimic and Libero.

Task	Pretrained Performance		Best-of- N (100 Rollouts)	
	BC	POSTBC	BC	POSTBC
Pick up banana	2/20	4/20	10/20	16/20
Put corn in pot	3/20	7/20	5/20	13/20

Table 3. Comparison of BC and POSTBC on real-world WidowX tasks with Best-of- N sampling. POSTBC enables significantly larger improvement, while preserving pretrained performance.

BC rollout + BC test	BC rollout + POSTBC test	POSTBC rollout + BC test	POSTBC rollout + POSTBC test
68.1 $\pm$ 2.2	82.7 $\pm$ 1.9	29.4 $\pm$ 3.6	81.3 $\pm$ 5.7

Table 4. Best-of- N sampling on Robomimic Lift with 2000 rollouts, varying the rollout policy used to collect data to train the Q -function, and the test-time (“test”) policy.

and Libero in Table 2. As these results illustrate, across both single-task and multi-task settings, the POSTBC policy performs comparably to, or even better than, the BC policy in terms of pretrained performance. Please see Table 5 for pretrained performance of σ -BC and VALUEDICE; in general, these approaches achieve pretrained performance comparable to or somewhat worse than that of BC. Combined with the results of Section 6.1, this shows that POSTBC enables more effective RL finetuning without hurting the performance of the pretrained policy.

6.3. POSTBC Scales to Real-World Settings

We next show that POSTBC scales to real-world robotic settings, leading to improvement in real-world RL finetuning over standard BC pretraining. We evaluate on the WidowX 250 6-DoF robot arm and consider two tasks on the scene in Figure 6. We first collect 10 human teleoperation demonstrations for the task “Put corn in pot”, where the objective is to pick up the corn and set it in the pot.



Figure 6. WidowX setup.

We train diffusion policies with standard BC as well as POSTBC on these demonstrations. For RL finetuning we consider two tasks—the original “Put corn in pot” task, and the task “Pick up banana” where the corn is replaced with a banana and the goal of the robot is to pick up the banana (see Figure 9). For RL finetuning, we consider the Best-of- N procedure outlined above, and utilize 100 rollouts per task.

Our results are given in Table 3. We see that POSTBC leads to significant improvements over BC—in both tasks achieving significantly higher final success rate. In particular, for the “Put corn in pot” task, RL finetuning only marginally improves the performance of the BC policy, while the POSTBC policy leads to significant improve-

ment. Furthermore, POSTBC pretraining actually leads to improved pretrained performance compared to the BC policy. This illustrates that POSTBC scales to real robot settings, providing improved RL finetuning performance without decreasing pretrained performance.

6.4. Understanding POSTBC

Finally, we seek to provide insight into how POSTBC improves RL finetuning performance (please see Section D.4 for additional ablations). We first aim to disambiguate the role of the exploration a POSTBC policy may provide over a BC policy, versus the role that having access to a larger action distribution at test time might play. For Best-of- N sampling, we can decouple this by selecting the rollout policy that collects data to learn the Q -function with IQL, and the policy whose actions we sample from and filter with the learned Q -function at test-time. We consider mixing the role of the BC and POSTBC policy on Robomimic Lift in Table 4. We find that using POSTBC as the test-time policy is critical to achieving effective performance, but that this performance is achievable whether we use BC or POSTBC for the rollout policy. This suggests that the utility of POSTBC is primarily in its ability to provide a wider range of actions that can be sampled at test time.

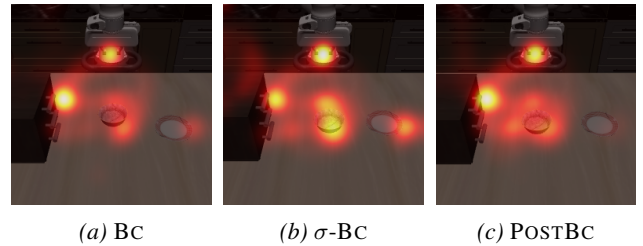


Figure 7. Qualitative analysis of POSTBC on Libero.

Next we consider the qualitative behavior of the POSTBC pretrained policy compared to the BC and σ -BC policies.

We illustrate this on Libero task “Open the top drawer of the cabinet and put the bowl in it” and display a heatmap of the visitations for each policy in Figure 7 (please see Section D.4 for visualizations on additional tasks). We see that POSTBC exhibits the widest distribution of states around the bowl, ensuring that it covers the behaviors necessary to reliably pick up the bowl, the most challenging aspect of this task. In contrast, BC and σ -BC exhibit less diversity around the bowl, giving them only a narrow set of behaviors to finetune from. At the same time, while exhibiting diversity around the states relevant to the task, POSTBC focuses its behavior only on these relevant behaviors, ensuring the pretrained policy still performs effectively. In contrast, σ -BC also interacts with the plate, which is irrelevant to this task and would therefore hurt its pretrained performance.

7. Conclusion

In this work, we have proposed a novel approach to pre-training policies from demonstrations that ensures the pre-trained performance is no worse than that of the BC policy, while expanding the action distribution to enable more effective RL finetuning. We have shown that this approach does indeed lead to improved RL finetuning performance in practice, scaling to real-world robotic settings. We believe this work motivates a variety of interesting questions for future work.

- Our demonstrator action coverage condition introduced in Section 4.1 is a *necessary* condition, in some cases, for RL finetuning to reach the performance of the demonstrator policy, as Proposition 2 shows. In general, however, demonstrator action coverage does not give a guarantee about the sample complexity of the downstream RL finetuning. Can we derive a non-trivial *sufficient* condition that ensures efficient RL finetuning without the aid of exploration approaches typically absent in practice (such as optimism), and how can we pretrain policies to ensure they meet such a sufficient condition?
- We have focused on pretraining only with supervised learning. While this is the most scalable approach, and the most commonly used approach in practice, is this a limiting factor in obtaining an effective initialization for online RL finetuning, and could we pretrain using other approaches as well (for example, offline RL)?
- While we have primarily considered applications to robotic control, our approach could also be applied in language domains. Does pretraining (or SFT finetuning) of language models with our approach lead to improved performance in downstream RL finetuning?

Acknowledgments

This research was partly supported by RAI, ONR N00014-25-1-2060, and NSF IIS-2150826. The work of CF was partially supported by an NSF CAREER award.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Abbeel, P. and Ng, A. Y. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 1, 2004.
- Agrawal, S. and Goyal, N. Analysis of thompson sampling for the multi-armed bandit problem. In *Conference on learning theory*, pp. 39–1. JMLR Workshop and Conference Proceedings, 2012.
- Ankile, L., Simeonov, A., Shenfeld, I., and Agrawal, P. Juicer: Data-efficient imitation learning for robotic assembly. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5096–5103. IEEE, 2024a.
- Ankile, L., Simeonov, A., Shenfeld, I., Torne, M., and Agrawal, P. From imitation to refinement—residual rl for precise assembly. *arXiv preprint arXiv:2407.16677*, 2024b.
- Azar, M. G., Guo, Z. D., Piot, B., Munos, R., Rowland, M., Valko, M., and Calandriello, D. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, pp. 4447–4455. PMLR, 2024.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., Das-Sarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022a.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022b.
- Bakker, M., Chadwick, M., Sheahan, H., Tessler, M., Campbell-Gillingham, L., Balaguer, J., McAleese, N.,

- Glaese, A., Aslanides, J., Botvinick, M., et al. Fine-tuning language models to find agreement among humans with diverse preferences. *Advances in Neural Information Processing Systems*, 35:38176–38189, 2022.
- Ball, P. J., Smith, L., Kostrikov, I., and Levine, S. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pp. 1577–1594. PMLR, 2023.
- Bellemare, M., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., and Munos, R. Unifying count-based exploration and intrinsic motivation. *Advances in neural information processing systems*, 29, 2016.
- Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Bojarski, M. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.
- Chae, J., Han, S., Jung, W., Cho, M., Choi, S., and Sung, Y. Robust imitation learning against variations in environment dynamics. In *International Conference on Machine Learning*, pp. 2828–2852. PMLR, 2022.
- Chakraborty, S., Qiu, J., Yuan, H., Koppel, A., Huang, F., Manocha, D., Bedi, A., and Wang, M. Maxmin-rlhf: Towards equitable alignment of large language models with diverse human preferences. In *ICML 2024 Workshop on Models of Human Feedback for AI Alignment*, 2024.
- Chang, J. D., Zhan, W., Oertell, O., Brantley, K., Misra, D., Lee, J. D., and Sun, W. Dataset reset policy optimization for rlhf. *arXiv preprint arXiv:2404.08495*, 2024.
- Chen, F., Huang, A., Golowich, N., Malladi, S., Block, A., Ash, J. T., Krishnamurthy, A., and Foster, D. J. The coverage principle: How pre-training enables post-training. *arXiv preprint arXiv:2510.15020*, 2025a.
- Chen, F., Raventos, A., Cheng, N., Ganguli, S., and Druckmann, S. Rethinking fine-tuning when scaling test-time compute: Limiting confidence improves mathematical reasoning. *arXiv preprint arXiv:2502.07154*, 2025b.
- Chen, H., Lu, C., Ying, C., Su, H., and Zhu, J. Offline reinforcement learning via high-fidelity generative behavior modeling. *arXiv preprint arXiv:2209.14548*, 2022.
- Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., and Zhao, D. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025c.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, pp. 02783649241273668, 2023.
- Choi, J., Guo, Y., Moczulski, M., Oh, J., Wu, N., Norouzi, M., and Lee, H. Contingency-aware exploration in reinforcement learning. *arXiv preprint arXiv:1811.01483*, 2018.
- Dasari, S. and Gupta, A. Transformers for one-shot visual imitation. In *Conference on Robot Learning*, pp. 2071–2084. PMLR, 2021.
- Dasari, S., Mees, O., Zhao, S., Srirama, M. K., and Levine, S. The ingredients for robotic diffusion transformers. *arXiv preprint arXiv:2410.10088*, 2024.
- Desai, S., Durugkar, I., Karnan, H., Warnell, G., Hanna, J., and Stone, P. An imitation from observation approach to transfer learning with dynamics mismatch. *Advances in Neural Information Processing Systems*, 33:3917–3929, 2020.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Dong, P., Li, Q., Sadigh, D., and Finn, C. Expo: Stable reinforcement learning with expressive policies. *arXiv preprint arXiv:2507.07986*, 2025a.
- Dong, P., Mirchandani, S., Sadigh, D., and Finn, C. What matters for batch online reinforcement learning in robotics? *arXiv preprint arXiv:2505.08078*, 2025b.
- Duan, Y., Schulman, J., Chen, X., Bartlett, P. L., Sutskever, I., and Abbeel, P. RI^2 : Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.
- Duan, Y., Andrychowicz, M., Stadie, B., Jonathan Ho, O., Schneider, J., Sutskever, I., Abbeel, P., and Zaremba, W. One-shot imitation learning. *Advances in neural information processing systems*, 30, 2017.

- Dumoulin, V., Johnson, D. D., Castro, P. S., Larochelle, H., and Dauphin, Y. A density estimation perspective on learning from pairwise human preferences. *arXiv preprint arXiv:2311.14115*, 2023.
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pp. 1126–1135. PMLR, 2017a.
- Finn, C., Yu, T., Zhang, T., Abbeel, P., and Levine, S. One-shot visual imitation learning via meta-learning. In *Conference on robot learning*, pp. 357–368. PMLR, 2017b.
- Finn, C., Xu, K., and Levine, S. Probabilistic model-agnostic meta-learning. *Advances in neural information processing systems*, 31, 2018.
- Foster, D. J., Kakade, S. M., Qian, J., and Rakhlin, A. The statistical complexity of interactive decision making. *arXiv preprint arXiv:2112.13487*, 2021.
- Fu, J., Luo, K., and Levine, S. Learning robust rewards with adversarial inverse reinforcement learning. *arXiv preprint arXiv:1710.11248*, 2017.
- Fushiki, T., Komaki, F., and Aihara, K. Nonparametric bootstrap prediction. *Bernoulli*, 11(2):293–307, 2005.
- Gao, C., Jiang, Y., and Chen, F. Transferring hierarchical structures with dual meta imitation learning. In *Conference on Robot Learning*, pp. 762–773. PMLR, 2023.
- Garg, D., Chakraborty, S., Cundy, C., Song, J., and Ermon, S. Iq-learn: Inverse soft-q learning for imitation. *Advances in Neural Information Processing Systems*, 34: 4028–4039, 2021.
- Ghosh, D., Ajay, A., Agrawal, P., and Levine, S. Offline rl policies should be trained to be adaptive. In *International Conference on Machine Learning*, pp. 7513–7530. PMLR, 2022.
- Giammarino, V., Queeney, J., and Paschalidis, I. C. Visually robust adversarial imitation learning from videos with contrastive learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15642–15648. IEEE, 2025.
- Gu, J., Kirmani, S., Wohlhart, P., Lu, Y., Arenas, M. G., Rao, K., Yu, W., Fu, C., Gopalakrishnan, K., Xu, Z., et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- Guo, Y., Zhang, J., Chen, X., Ji, X., Wang, Y.-J., Hu, Y., and Chen, J. Improving vision-language-action model with online reinforcement learning. *arXiv preprint arXiv:2501.16664*, 2025b.
- Hansen-Estruch, P., Kostrikov, I., Janner, M., Kuba, J. G., and Levine, S. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- He, L., Shen, L., Tan, J., and Wang, X. Aligniql: Policy alignment in implicit q-learning through constrained optimization. *arXiv preprint arXiv:2405.18187*, 2024.
- Henaff, M., Raileanu, R., Jiang, M., and Rocktäschel, T. Exploration via elliptical episodic bonuses. *Advances in Neural Information Processing Systems*, 35:37631–37646, 2022.
- Ho, J. and Ermon, S. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29, 2016.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Hu, H., Yang, Y., Ye, J., Mai, Z., and Zhang, C. Unsupervised behavior extraction via random intent priors. *Advances in Neural Information Processing Systems*, 36: 51491–51514, 2023.
- Hu, J., Hendrix, R., Farhadi, A., Kembhavi, A., Martín-Martín, R., Stone, P., Zeng, K.-H., and Ehsani, K. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3617–3624. IEEE, 2025.
- James, S., Bloesch, M., and Davison, A. J. Task-embedded control networks for few-shot imitation learning. In *Conference on robot learning*, pp. 783–795. PMLR, 2018.
- Janz, D., Liu, S., Ayoub, A., and Szepesvári, C. Exploration via linearly perturbed loss minimisation. In *International Conference on Artificial Intelligence and Statistics*, pp. 721–729. PMLR, 2024.
- Jin, H., Luan, S., Lyu, S., Rabusseau, G., Rabbany, R., Precup, D., and Hamdaqa, M. RL fine-tuning heals ood forgetting in sft. *arXiv preprint arXiv:2509.12235*, 2025.

- Jülg, T., Burgard, W., and Walter, F. Refined policy distillation: From vla generalists to rl experts. *arXiv preprint arXiv:2503.05833*, 2025.
- Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., Fagan, P. D., Hejna, J., Itkina, M., Lepert, M., Ma, Y. J., Miller, P. T., Wu, J., Belkhale, S., Dass, S., Ha, H., Jain, A., Lee, A., Lee, Y., Memmel, M., Park, S., Radosavovic, I., Wang, K., Zhan, A., Black, K., Chi, C., Hatch, K. B., Lin, S., Lu, J., Mercat, J., Rehman, A., Sanketi, P. R., Sharma, A., Simpson, C., Vuong, Q., Walke, H. R., Wulfe, B., Xiao, T., Yang, J. H., Yavary, A., Zhao, T. Z., Agia, C., Baijal, R., Castro, M. G., Chen, D., Chen, Q., Chung, T., Drake, J., Foster, E. P., Gao, J., Guizilini, V., Herrera, D. A., Heo, M., Hsu, K., Hu, J., Irshad, M. Z., Jackson, D., Le, C., Li, Y., Lin, K., Lin, R., Ma, Z., Maddukuri, A., Mirchandani, S., Morton, D., Nguyen, T., O’Neill, A., Scalise, R., Seale, D., Son, V., Tian, S., Tran, E., Wang, A. E., Wu, Y., Xie, A., Yang, J., Yin, P., Zhang, Y., Bastani, O., Berseth, G., Bohg, J., Goldberg, K., Gupta, A., Gupta, A., Jayaraman, D., Lim, J. J., Malik, J., Martín-Martín, R., Ramamoorthy, S., Sadigh, D., Song, S., Wu, J., Yip, M. C., Zhu, Y., Kollar, T., Levine, S., and Finn, C. Droid: A large-scale in-the-wild robot manipulation dataset. 2024.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Kostrikov, I., Agrawal, K. K., Dwibedi, D., Levine, S., and Tompson, J. Discriminator-actor-critic: Addressing sample inefficiency and reward bias in adversarial imitation learning. *arXiv preprint arXiv:1809.02925*, 2018.
- Kostrikov, I., Nachum, O., and Tompson, J. Imitation learning via off-policy distribution matching. *arXiv preprint arXiv:1912.05032*, 2019.
- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169*, 2021.
- Kumar, A., Singh, A., Ebert, F., Nakamoto, M., Yang, Y., Finn, C., and Levine, S. Pre-training for robots: Offline rl enables learning new tasks from a handful of trials. *arXiv preprint arXiv:2210.05178*, 2022.
- Kveton, B., Zaheer, M., Szepesvari, C., Li, L., Ghavamzadeh, M., and Boutilier, C. Randomized exploration in generalized linear bandits. In *International Conference on Artificial Intelligence and Statistics*, pp. 2066–2076. PMLR, 2020.
- Lee, H., Phatale, S., Mansoor, H., Lu, K., Mesnard, T., Bishop, C., Carbune, V., and Rastogi, A. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.
- Lee, K., Laskin, M., Srinivas, A., and Abbeel, P. Sunrise: A simple unified framework for ensemble learning in deep reinforcement learning. In *International Conference on Machine Learning*, pp. 6131–6141. PMLR, 2021.
- Lee, S., Seo, Y., Lee, K., Abbeel, P., and Shin, J. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*, pp. 1702–1712. PMLR, 2022.
- Li, Q., Zhang, J., Ghosh, D., Zhang, A., and Levine, S. Accelerating exploration with unlabeled prior data. *Advances in Neural Information Processing Systems*, 36: 67434–67458, 2023a.
- Li, Z., Xu, T., Qin, Z., Yu, Y., and Luo, Z.-Q. Imitation learning from imperfection: Theoretical justifications and algorithms. *Advances in Neural Information Processing Systems*, 36:18404–18443, 2023b.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Liu, J., Gao, F., Wei, B., Chen, X., Liao, Q., Wu, Y., Yu, C., and Wang, Y. What can rl bring to vla generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025.
- Lu, G., Guo, W., Zhang, C., Zhou, Y., Jiang, H., Gao, Z., Tang, Y., and Wang, Z. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.
- Mandlekar, A., Xu, D., Wong, J., Nasiriany, S., Wang, C., Kulkarni, R., Fei-Fei, L., Savarese, S., Zhu, Y., and Martín-Martín, R. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.
- Mark, M. S., Gao, T., Sampaio, G. G., Srirama, M. K., Sharma, A., Finn, C., and Kumar, A. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.
- Munos, R., Valko, M., Calandriello, D., Azar, M. G., Rowland, M., Guo, Z. D., Tang, Y., Geist, M., Mesnard, T., Michi, A., et al. Nash learning from human feedback. *arXiv preprint arXiv:2312.00886*, 2023.

- Nakamoto, M., Zhai, S., Singh, A., Sobol Mark, M., Ma, Y., Finn, C., Kumar, A., and Levine, S. Cal-ql: Calibrated offline rl pre-training for efficient online finetuning. *Advances in Neural Information Processing Systems*, 36:62244–62269, 2023.
- Nakamoto, M., Mees, O., Kumar, A., and Levine, S. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.
- Ng, A. Y., Russell, S., et al. Algorithms for inverse reinforcement learning. In *Icml*, volume 1, pp. 2, 2000.
- Ni, T., Sikchi, H., Wang, Y., Gupta, T., Lee, L., and Eysenbach, B. f-irl: Inverse reinforcement learning via state marginal matching. In *Conference on Robot Learning*, pp. 529–551. PMLR, 2021.
- Osband, I. and Van Roy, B. Bootstrapped thompson sampling and deep exploration. *arXiv preprint arXiv:1507.00300*, 2015.
- Osband, I., Blundell, C., Pritzel, A., and Van Roy, B. Deep exploration via bootstrapped dqn. *Advances in neural information processing systems*, 29, 2016a.
- Osband, I., Van Roy, B., and Wen, Z. Generalization and exploration via randomized value functions. In *International Conference on Machine Learning*, pp. 2377–2386. PMLR, 2016b.
- Osband, I., Aslanides, J., and Cassirer, A. Randomized prior functions for deep reinforcement learning. *Advances in neural information processing systems*, 31, 2018.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741, 2023.
- Rahmatizadeh, R., Abolghasemi, P., Bölöni, L., and Levine, S. Vision-based multi-task manipulation for inexpensive robots using end-to-end learning from demonstration. In *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 3758–3765. IEEE, 2018.
- Rajaraman, N., Yang, L., Jiao, J., and Ramchandran, K. Toward the fundamental limits of imitation learning. *Advances in Neural Information Processing Systems*, 33: 2914–2924, 2020.
- Ramamurthy, R., Ammanabrolu, P., Brantley, K., Hessel, J., Sifa, R., Bauckhage, C., Hajishirzi, H., and Choi, Y. Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization. *arXiv preprint arXiv:2210.01241*, 2022.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.
- Rosset, C., Cheng, C.-A., Mitra, A., Santacroce, M., Awadallah, A., and Xie, T. Direct nash optimization: Teaching language models to self-improve with general preferences. *arXiv preprint arXiv:2404.03715*, 2024.
- Russo, D. Worst-case regret bounds for exploration via randomized value functions. *Advances in neural information processing systems*, 32, 2019.
- Russo, D. and Van Roy, B. Learning to optimize via posterior sampling. *Mathematics of Operations Research*, 39 (4):1221–1243, 2014.
- Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., Wen, Z., et al. A tutorial on thompson sampling. *Foundations and Trends® in Machine Learning*, 11(1):1–96, 2018.
- Shafiullah, N. M., Cui, Z., Altanzaya, A. A., and Pinto, L. Behavior transformers: Cloning k modes with one stone. *Advances in neural information processing systems*, 35: 22955–22968, 2022.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shyam, P., Jaśkowski, W., and Gomez, F. Model-based active exploration. In *International conference on machine learning*, pp. 5779–5788. PMLR, 2019.
- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pp. 2256–2265. pmlr, 2015.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

- Springer, J. M., Goyal, S., Wen, K., Kumar, T., Yue, X., Malladi, S., Neubig, G., and Raghunathan, A. Over-trained language models are harder to fine-tune. *arXiv preprint arXiv:2503.19206*, 2025.
- Sridhar, A., Shah, D., Glossop, C., and Levine, S. No-mad: Goal masked diffusion policies for navigation and exploration. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 63–70. IEEE, 2024.
- Stadie, B. C., Levine, S., and Abbeel, P. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv preprint arXiv:1507.00814*, 2015.
- Stepputtis, S., Campbell, J., Phielipp, M., Lee, S., Baral, C., and Ben Amor, H. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.
- Swamy, G., Dann, C., Kidambi, R., Wu, Z. S., and Agarwal, A. A minimaximalist approach to reinforcement learning from human feedback. *arXiv preprint arXiv:2401.04056*, 2024.
- Tang, Y., Guo, Z. D., Zheng, Z., Calandriello, D., Munos, R., Rowland, M., Richemond, P. H., Valko, M., Pires, B. Á., and Piot, B. Generalized preference optimization: A unified approach to offline alignment. *arXiv preprint arXiv:2402.05749*, 2024.
- Tangkaratt, V., Charoenphakdee, N., and Sugiyama, M. Robust imitation learning from noisy demonstrations. *arXiv preprint arXiv:2010.10181*, 2020.
- Team, K., Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Team, O. M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Thompson, W. R. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Uchendu, I., Xiao, T., Lu, Y., Zhu, B., Yan, M., Simon, J., Bennice, M., Fu, C., Ma, C., Jiao, J., et al. Jump-start reinforcement learning. In *International Conference on Machine Learning*, pp. 34556–34583. PMLR, 2023.
- Wagenmaker, A., Nakamoto, M., Zhang, Y., Park, S., Yagoub, W., Nagabandi, A., Gupta, A., and Levine, S. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025a.
- Wagenmaker, A., Zhou, Z., and Levine, S. Behavioral exploration: Learning to explore via in-context adaptation. In *Forty-second International Conference on Machine Learning*, 2025b.
- Walke, H. R., Black, K., Zhao, T. Z., Vuong, Q., Zheng, C., Hansen-Estruch, P., He, A. W., Myers, V., Kim, M. J., Du, M., et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pp. 1723–1736. PMLR, 2023.
- Wang, J. X., Kurth-Nelson, Z., Tirumala, D., Soyer, H., Leibo, J. Z., Munos, R., Blundell, C., Kumaran, D., and Botvinick, M. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.
- Wang, Y., Xu, C., and Du, B. Robust adversarial imitation learning via adaptively-selected demonstrations. In *IJCAI*, pp. 3155–3161, 2021.
- Wilcoxson, M., Li, Q., Frans, K., and Levine, S. Leveraging skills from unlabeled prior data for efficient online exploration. *arXiv preprint arXiv:2410.18076*, 2024.
- Xu, C., Li, Q., Luo, J., and Levine, S. Rldg: Robotic generalist policy distillation via reinforcement learning. *arXiv preprint arXiv:2412.09858*, 2024.
- Xu, H., Zhan, X., Yin, H., and Qin, H. Discriminator-weighted offline imitation learning from suboptimal demonstrations. In *International Conference on Machine Learning*, pp. 24725–24742. PMLR, 2022.
- Yin, Y., Wang, Z., Gu, Y., Huang, H., Chen, W., and Zhou, M. Relative preference optimization: Enhancing llm alignment through contrasting responses across identical and diverse prompts. *arXiv preprint arXiv:2402.10958*, 2024.
- Yuan, X., Mu, T., Tao, S., Fang, Y., Zhang, M., and Su, H. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024.
- Yue, S., Hua, X., Ren, J., Lin, S., Zhang, J., and Zhang, Y. Ollie: Imitation learning from offline pretraining to online finetuning. *arXiv preprint arXiv:2405.17477*, 2024.

- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., and Xu, H. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.
- Zeng, H., Hui, K., Zhuang, H., Qin, Z., Yue, Z., Zamani, H., and Alon, D. Can pre-training indicators reliably predict fine-tuning outcomes of llms? *arXiv preprint arXiv:2504.12491*, 2025.
- Zhang, H., Xu, W., and Yu, H. Policy expansion for bridging offline-to-online reinforcement learning. *arXiv preprint arXiv:2302.00935*, 2023.
- Zhang, T., McCarthy, Z., Jow, O., Lee, D., Chen, X., Goldberg, K., and Abbeel, P. Deep imitation learning for complex manipulation tasks from virtual reality teleoperation. In *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 5628–5635. IEEE, 2018.
- Zhang, Z., Zheng, K., Chen, Z., Jang, J., Li, Y., Han, S., Wang, C., Ding, M., Fox, D., and Yao, H. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.
- Zhao, T. Z., Tompson, J., Driess, D., Florence, P., Ghasemipour, K., Finn, C., and Wahid, A. Aloha unleashed: A simple recipe for robot dexterity. *arXiv preprint arXiv:2410.13126*, 2024.
- Zheng, H., Luo, X., Wei, P., Song, X., Li, D., and Jiang, J. Adaptive policy learning for offline-to-online reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 11372–11380, 2023.
- Ziebart, B. D., Maas, A. L., Bagnell, J. A., Dey, A. K., et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pp. 1433–1438. Chicago, IL, USA, 2008.
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., and Irving, G. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- Zintgraf, L., Shiarlis, K., Igl, M., Schulze, S., Gal, Y., Hofmann, K., and Whiteson, S. Varibad: A very good method for bayes-adaptive deep rl via meta-learning. *arXiv preprint arXiv:1910.08348*, 2019.

A. Additional Related Work

RL finetuning of pretrained policies. RL finetuning of pretrained policies is a critical step in both language and robotic domains. In language domains, RL finetuning has proved crucial in aligning LLMs to human values (Ziegler et al., 2019; Ouyang et al., 2022; Bai et al., 2022a; Ramamurthy et al., 2022; Touvron et al., 2023), and enabling reasoning abilities (Shao et al., 2024; Team et al., 2025; Guo et al., 2025a). A host of finetuning algorithms have been developed, both online (Bai et al., 2022b; Bakker et al., 2022; Dumoulin et al., 2023; Lee et al., 2023; Munos et al., 2023; Swamy et al., 2024; Chakraborty et al., 2024; Chang et al., 2024) and offline (Rafailov et al., 2023; Azar et al., 2024; Rosset et al., 2024; Tang et al., 2024; Yin et al., 2024). In robotic control domains, RL finetuning methods include directly modifying the weights of the base pretrained policy (Zhang et al., 2024; Xu et al., 2024; Mark et al., 2024; Ren et al., 2024; Hu et al., 2025; Guo et al., 2025b; Lu et al., 2025; Chen et al., 2025c; Liu et al., 2025), Best-of- N sampling-style approaches that filter the output of the pretrained policy with a learned value function (Chen et al., 2022; Hansen-Estruch et al., 2023; He et al., 2024; Nakamoto et al., 2024; Dong et al., 2025b), “steering” the pretrained policy by altering its sampling process (Wagenmaker et al., 2025a), and learning smaller residual policies to augment the pretrained policy’s actions (Ankile et al., 2024b; Yuan et al., 2024; Jülg et al., 2025; Dong et al., 2025a). Our work is tangential to this line of work: rather than improving the finetuning algorithm, we aim to ensure the pretrained policy is amenable to RL finetuning.

Imitation learning and meta-learning. While our primary focus is on behavioral cloning (as noted, the workhorse of most modern applications) other approaches to pretraining from demonstrations exist. BC is only one possible instantiation of *imitation learning*; other approaches to imitation learning include inverse RL (Ng et al., 2000; Abbeel & Ng, 2004; Ziebart et al., 2008), methods that aim to learn a policy matching the state distribution of the demonstrator, such as adversarial imitation learning (Ho & Ermon, 2016; Kostrikov et al., 2018; Fu et al., 2017; Kostrikov et al., 2019; Ni et al., 2021; Garg et al., 2021; Xu et al., 2022; Li et al., 2023b; Yue et al., 2024), and robust imitation learning (Chae et al., 2022; Desai et al., 2020; Tangkaratt et al., 2020; Wang et al., 2021; Giammarino et al., 2025). The majority of these works, however, either assume access to additional data sources (e.g. suboptimal trajectories), or require online environment access and are therefore not truly offline pretraining approaches, which is the focus of this work. Furthermore, none of these works explicitly consider the role of pretraining in enabling efficient RL finetuning.

Meta-learning directly aims learn an initialization that can be quickly adapted to a new task. While instantiations of meta-learning for imitation learning exist (Duan et al., 2017; Finn et al., 2017b; James et al., 2018; Dasari & Gupta, 2021; Gao et al., 2023), our setting differs fundamentally from the meta-imitation learning setting. Meta-imitation learning assumes access to demonstration data from *more than one task*, and attempts to learn an initialization that will allow for quickly adapting to demonstrations from a *new* task. In contrast, our goal is to obtain an approach able to learn on a one (or potentially multiple) task(s), and we aim to find an initialization that allows for improvement on the *same* task (or set of tasks), while preserving pretrained performance on this task. Furthermore, rather than learning from new *demonstrations*, as meta-imitation learning does, we aim to learn from (potentially suboptimal) data collected online and that is labeled with rewards. Finally, a primary goal of our work is to ensure that the pretrained policy itself performs well (that is, it has effective 0-shot performance, on par with BC). Meta-imitation learning, in contrast, typically does not consider the 0-shot performance of the pretrained policy, but instead considers the 1-shot performance, i.e. the performance after it has already been updated on the new task. This is a fundamentally different constraint on the pretrained policy than is typically present in the meta-learning setting, but is critical in real-world settings where we want a pretrained policy that behaves well 0-shot.

Reinforcement learning-based pretraining. In the RL literature, two lines of work bear some resemblance to ours as well. The *offline-to-online RL* setting aims to train policies with RL on offline datasets that can then be improved with further online interaction (Lee et al., 2022; Ghosh et al., 2022; Kumar et al., 2022; Zhang et al., 2023; Uchendu et al., 2023; Zheng et al., 2023; Ball et al., 2023; Nakamoto et al., 2023), and the *meta-RL* setting aims to meta-learn a policy on some set of tasks which can then be quickly adapted to a new task (Wang et al., 2016; Duan et al., 2016; Finn et al., 2017a; 2018). While similar to our work in that these works also aim to learn behaviors that can be efficiently improved online, the settings differ significantly in that the offline- or meta-pretraining typically requires reward labels (rather than unlabeled demonstrations) and are performed with RL (rather than BC)—in contrast, we study how BC-like pretraining (as noted, the workhorse of most modern applications) can enable efficient online adaptation.

Posterior sampling and exploration. Our proposed approach relies on modeling the posterior distribution of the demonstrator’s behavior. While this is, to the best of our knowledge, the first example of applying posterior sampling to

BC, posterior methods have a long history in RL, going back to the work of (Thompson, 1933). This works spans applied (Osband et al., 2016a;b; 2018; Zintgraf et al., 2019) and theoretical (Agrawal & Goyal, 2012; Russo & Van Roy, 2014; Russo et al., 2018; Janz et al., 2024; Kveton et al., 2020; Russo, 2019) settings. More generally, our approach can be seen as enabling BC-trained policies to *explore* more effectively. Exploration is a well-studied problem in the RL community (Stadie et al., 2015; Bellemare et al., 2016; Burda et al., 2018; Choi et al., 2018; Ecoffet et al., 2019; Shyam et al., 2019; Lee et al., 2021; Henaff et al., 2022), with several works considering learning exploration strategies from offline data (Hu et al., 2023; Li et al., 2023a; Wilcoxon et al., 2024; Wagenmaker et al., 2025b). These works, however, either consider RL-based pretraining (while we focus on BC) or do not consider the question of online finetuning.

B. Proofs

B.1. BC Policy Fails to Cover Demonstrator Actions

Proof of Proposition 2. Let $\mathcal{M}^1$ and $\mathcal{M}^2$ denote multi-armed bandits with 3 arms and reward functions r^1 and r^2 :

$$\begin{aligned} r^1(a_1) &= 0, r^1(a_2) = 1, r^1(a_3) = 0 \\ r^2(a_1) &= 0, r^2(a_2) = 0, r^2(a_3) = 1. \end{aligned}$$

Let $\pi^\beta(a_1) = 1 - 4\epsilon$, $\pi^\beta(a_2) = 2\epsilon$, $\pi^\beta(a_3) = 2\epsilon$.

By construction of $\hat{\pi}^{\text{bc}}$, if $T(a_2) = 0$ then we will have $\hat{\pi}^{\text{bc}}(a_2) = 0$, and if $T(a_3) = 0$ we will have $\hat{\pi}^{\text{bc}}(a_3) = 0$. By the definition of both $\mathcal{M}^1$ and $\mathcal{M}^2$, we have

$$\mathbb{P}^{\mathcal{M}^i}[T(a_2) = 0, T(a_3) = 0] = (1 - 4\epsilon)^T.$$

As we have assumed that $T \leq \frac{1}{20\epsilon}$ and $\epsilon \in (0, 1/8]$, some calculation shows that we can lower bound this as $1/2$. Note that for both $\mathcal{M}^1$ and $\mathcal{M}^2$, we have $\mathcal{J}(\pi^\beta) = 2\epsilon$, while for policies $\hat{\pi}^{\text{bc}}$ that only play a_1 , we have $\mathcal{J}(\hat{\pi}^{\text{bc}}) = 0$. This proves the first part of the result.

For the second part, note that the optimal policy on $\mathcal{M}^1$ plays only a_2 and has expected reward of 1, while the optimal policy on $\mathcal{M}^2$ plays only a_3 and has expected reward of 1. Let $\hat{\pi}$ denote an estimate of the optimal policy and $\mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}}[\cdot]$ the expectation induced by playing the policy $\hat{\pi}^{\text{bc}}$ from the first part on instance $\mathcal{M}^i$. Then:

$$\min_{\hat{\pi}} \max_{i \in \{1,2\}} \mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}}[\max_{\pi} \mathcal{J}^{\mathcal{M}^i}(\pi) - \mathcal{J}^{\mathcal{M}^i}(\hat{\pi})] = \min_{\hat{\pi}} \max_{i \in \{1,2\}} \mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}}[1 - \hat{\pi}(a_{1+i})].$$

Note that $1 - \hat{\pi}(a_2) = \hat{\pi}(a_1) + \hat{\pi}(a_3) \geq \hat{\pi}(a_3)$. Thus we can lower bound the above as

$$\begin{aligned} &\geq \min_{\hat{\pi}} \max\{\mathbb{E}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)], \mathbb{E}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}[1 - \hat{\pi}(a_3)]\} \\ &\geq \min_{\hat{\pi}} \frac{1}{2} \left(\mathbb{E}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)] + \mathbb{E}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}[1 - \hat{\pi}(a_3)] \right) \\ &\geq \frac{1}{2} - \frac{1}{2} \min_{\hat{\pi}} \left| \mathbb{E}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)] - \mathbb{E}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)] \right|. \end{aligned}$$

We can bound

$$\left| \mathbb{E}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)] - \mathbb{E}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}[\hat{\pi}(a_3)] \right| \leq \text{TV}(\mathbb{P}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}, \mathbb{P}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}).$$

Since $\mathcal{M}^1$ and $\mathcal{M}^2$ only differ on a_2 and a_3 , and since $\hat{\pi}^{\text{bc}}(a_2) = \hat{\pi}^{\text{bc}}(a_3) = 0$, we have $\text{TV}(\mathbb{P}^{\mathcal{M}^1, \hat{\pi}^{\text{bc}}}, \mathbb{P}^{\mathcal{M}^2, \hat{\pi}^{\text{bc}}}) = 0$. Thus, we conclude that

$$\min_{\hat{\pi}} \max_{i \in \{1,2\}} \mathbb{E}^{\mathcal{M}^i, \hat{\pi}^{\text{bc}}}[\max_{\pi} \mathcal{J}^{\mathcal{M}^i}(\pi) - \mathcal{J}^{\mathcal{M}^i}(\hat{\pi})] \geq \frac{1}{2}.$$

This proves the second part of the result. $\square$

B.2. Uniform Noise Fails

Proof of Proposition 3. Construction. Let $\mathcal{M}$ be the MDP with state space $\{\tilde{s}_1, \dots, \tilde{s}_k, s_1, s_2\}$, actions $\{a_1, a_2\}$, horizon $H \geq 2$ with initial state distribution:

$$P_0(s_1) = 1/2, \quad P_0(\tilde{s}_1) = 2^{-2} + 2^{-k}, \quad P_0(\tilde{s}_i) = 2^{-i-1}, i \geq 2,$$

transition function, for all $h \in [H]$:

$$\begin{aligned} P_h(\tilde{s}_i | \tilde{s}_i, a) &= 1, \forall a \in \mathcal{A}, & P_h(s_1 | s_1, a_1) &= 1, \\ P_h(s_2 | s_1, a_2) &= 1, & P_h(s_2 | s_2, a) &= 1, \forall a \in \mathcal{A}, \end{aligned}$$

and reward that is 0 everywhere except

$$r_1(\tilde{s}_i, a_1) = r_H(s_1, a_1) = 1, \quad r_1(\tilde{s}_i, a_2) = 1 - 2\Delta,$$

for some $\Delta > 0$ to be specified. We consider π^β defined as

$$\pi_h^\beta(a_1 | \tilde{s}_i) = \pi_h^\beta(a_2 | \tilde{s}_i) = \frac{1}{2}, \quad \pi_h^\beta(a_1 | s_1) = 1.$$

Let $\epsilon := \frac{H^2 S \log T}{T} + \xi$, and set $\Delta \leftarrow 2\epsilon$.

Upper bound on α . Note that $\mathcal{J}(\pi^\beta) = 1 - \frac{1}{2}\Delta$, and that the value of the optimal policy π^* is $\mathcal{J}(\pi^*) = \max_\pi \mathcal{J}(\pi) = 1$. Let $\tilde{\pi}^{u,\alpha}$ denote the policy that, on all $\tilde{s}_i$ plays π^* , and on other states plays π^* with probability $1 - \alpha$, and otherwise plays $\text{unif}(\mathcal{A})$. Note then that, regardless of the value of $\hat{\pi}^{\text{bc}}$, we have that $\mathcal{J}(\tilde{\pi}^{u,\alpha}) \geq \mathcal{J}(\hat{\pi}^{u,\alpha})$. Thus,

$$\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{u,\alpha})] \geq \mathcal{J}(\pi^\beta) - \mathcal{J}(\tilde{\pi}^{u,\alpha})$$

If we are in s_1 at $h = 2$, the only way we can receive any reward on the episode is if we take action a_1 for the last $H - 1$ steps, and we then receive a reward of 1. Under $\tilde{\pi}^{u,\alpha}$, we take a_1 at each step with probability $1 - \alpha + \alpha/A$, so our probability of getting a reward of 1 is $(1 - \alpha + \alpha/A)^{H-1}$. Note that in contrast π^β will always play a_1 and receive a reward of 1 in this situation. If we are in $\tilde{s}_i$ at $h = 2$ for any i , then π^β will incur a loss of Δ more than $\tilde{\pi}^{u,\alpha}$. Thus, we can lower bound

$$\mathcal{J}(\pi^\beta) - \mathcal{J}(\tilde{\pi}^{u,\alpha}) \geq -\frac{1}{2}\Delta + \frac{1}{2} \cdot (1 - (1 - \alpha + \alpha/A)^{H-1})$$

By assumption we have that $\frac{1}{2}\Delta = \epsilon$. Thus, if we want $\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{u,\alpha})] \leq \epsilon$, we need

$$\frac{1}{2} \cdot (1 - (1 - \alpha + \alpha/A)^{H-1}) \leq 2\epsilon.$$

Rearranging this, we have

$$1 - 4\epsilon \leq (1 - \alpha + \alpha/A)^{H-1} \iff \frac{1}{H-1} \log(1 - 4\epsilon) \leq \log(1 - \alpha + \alpha/A).$$

From the Taylor decomposition of $\log(1 - x)$, we see that $\log(1 - \alpha + \alpha/A) \leq -(1 - 1/A)\alpha$. Furthermore, we can lower bound

$$\log(1 - 4\epsilon) \geq -8\epsilon$$

as long as $\epsilon \leq 1/2$. Altogether, then, we have

$$\frac{-8\epsilon}{H-1} \leq -(1 - 1/A)\alpha \implies \alpha \leq \frac{8\epsilon}{(H-1)(1 - 1/A)} \implies \alpha \leq 32\epsilon$$

where the last inequality follows since $H \geq 2, A = 2$.

Upper bound on γ . Let $i_T := \arg \max_i \{2^{-i-1} \mid 2^{-i-1} \leq 1/T\}$, so that $1/2T \leq P_0(\tilde{s}_{i_T}) \leq 1/T$, and note that such an $\tilde{s}_{i_T}$ exists by construction. Let $\mathcal{E}$ be the event $\mathcal{E} := \{T_1(\tilde{s}_{i_T}) = T_1(\tilde{s}_{i_T}, a_2) = 1\}$. We have

$$\begin{aligned} \mathbb{P}[\mathcal{E}] &= \mathbb{P}[T_1(\tilde{s}_{i_T}, a_2) = 1 \mid T_1(\tilde{s}_{i_T}) = 1] \mathbb{P}[T_1(\tilde{s}_{i_T}) = 1] \\ &= \frac{1}{2} \cdot T P_0(\tilde{s}_{i_T}) (1 - P_0(\tilde{s}_{i_T}))^{T-1} \\ &= \frac{1}{2} \cdot T \cdot \frac{1}{2T} \cdot (1 - \frac{1}{T})^{T-1} \\ &\geq \frac{1}{4e}. \end{aligned}$$

Note that on the event $\mathcal{E}$, we have $\hat{\pi}_1^{\text{bc}}(a_1 \mid \tilde{s}_{i_T}) = 0$, but $\pi_1^\beta(a_1 \mid \tilde{s}_{i_T}) = 1/2$. Thus,

$$\hat{\pi}_1^{\text{u},\alpha}(a_1 \mid \tilde{s}_{i_T}) = \alpha/A \leq 32\epsilon/A = 64\epsilon/A \cdot \pi_1^\beta(a_1 \mid \tilde{s}_{i_T})$$

where we have used the bound on α shown above. Thus, on $\mathcal{E}$, we will only have that $\hat{\pi}^{\text{u},\alpha}$ achieves demonstrator action coverage for $\gamma \leq 64\epsilon/A$. Since $\mathcal{E}$ occurs with probability at least $1/4e$, it follows that if we want to guarantee $\hat{\pi}^{\text{u},\alpha}$ achieves demonstrator action coverage with probability at least $1 - \delta$ for $\delta < 1/4e$, we must have $\gamma \leq 64\epsilon/A$.

Note as well that, since $\hat{\pi}_1^{\text{bc}}(a_2 \mid \tilde{s}_{i_T}) = 1$, any policy in the support of $\hat{\pi}^{\text{bc}}$ will be suboptimal by a factor of at least $P_0(\tilde{s}_{i_T}) \cdot 2\Delta \geq \Delta/T$. $\square$

B.3. Analysis of Posterior Demonstrator Policy

In the tabular setting, some algebra shows that

$$\hat{\pi}_h^{\text{post}}(a \mid s) = \begin{cases} \frac{T_h(s,a)+1}{T_h(s)+A} & T_h(s) > 0 \\ \text{unif}(\mathcal{A}) & T_h(s) = 0. \end{cases}$$

Throughout this section we denote

$$\tilde{\pi}_h(a \mid s) := \begin{cases} (1 - \alpha) \cdot \frac{T_h(s,a)}{T_h(s)} + \alpha \cdot \frac{T_h(s,a)+\lambda/A}{T_h(s)+\lambda} & T_h(s) > 0 \\ \text{unif}(\mathcal{A}) & T_h(s) = 0 \end{cases}$$

for some $\alpha \in [0, 1]$.

We also denote $w_h^\pi(s, a) := \mathbb{P}^\pi[s_h = s, a_h = a]$. $Q_h^\pi(s, a) := \mathbb{E}^\pi[\sum_{h' \geq h} r_{h'}(s_{h'}, a_{h'}) \mid s_h = s, a_h = a]$ denotes the standard Q -function. $\mathcal{J}(\pi; r)$ denotes the expected return of policy π for reward r .

Lemma 1. *As long as $\delta \leq 0.9$ and $\lambda \geq A$, we have*

$$\mathbb{P} \left[\tilde{\pi}_h(a \mid s) \geq \alpha \cdot \min \left\{ \frac{\pi_h^\beta(a \mid s)}{64 \log SH / \delta}, \frac{1}{2\lambda} \right\}, \forall a \in \mathcal{A}, s \in \mathcal{S}, h \in [H] \right] \geq 1 - \delta.$$

Proof. Consider some (s, h) . By Bernstein's inequality, if $T_h(s) > 0$, we have that with probability at least $1 - \delta$,

$$\frac{T_h(s,a)}{T_h(s)} \geq \pi_h^\beta(a \mid s) - \sqrt{\frac{2\pi_h^\beta(a \mid s) \log 1/\delta}{T_h(s)}} - \frac{2 \log 1/\delta}{3T_h(s)}. \quad (3)$$

From some algebra, we see that as long as $T_h(s) \geq \frac{32 \log 1/\delta}{\pi_h^\beta(a \mid s)}$, we have that $\frac{T_h(s,a)}{T_h(s)} \geq \frac{1}{2} \pi_h^\beta(a \mid s)$. By the definition of $\tilde{\pi}$,

under the good event of (3) we can then lower bound

$$\begin{aligned}
 \tilde{\pi}_h(a | s) &\geq \begin{cases} \frac{\alpha}{1+\lambda/T_h(s)} \cdot \frac{1}{2} \pi_h^\beta(a | s) & T_h(s) \geq \frac{32 \log 1/\delta}{\pi_h^\beta(a | s)} \\ \frac{\alpha \lambda / A}{T_h(s) + A} & \text{o.w.} \end{cases} \\
 &\geq \begin{cases} \frac{\alpha \cdot 32 \log 1/\delta}{32 \log 1/\delta + \lambda \cdot \pi_h^\beta(a | s)} \cdot \frac{1}{2} \pi_h^\beta(a | s) & N_h(s) \geq \frac{32 \log 1/\delta}{\pi_h^\beta(a | s)} \\ \frac{\alpha \lambda / A \cdot \pi_h^\beta(a | s)}{32 \log 1/\delta + \lambda \cdot \pi_h^\beta(a | s)} & \text{o.w.} \end{cases} \\
 &\stackrel{(a)}{\geq} \frac{\alpha \cdot \pi_h^\beta(a | s)}{32 \log 1/\delta + \lambda \cdot \pi_h^\beta(a | s)} \\
 &\geq \alpha \cdot \min \left\{ \frac{\pi_h^\beta(a | s)}{64 \log 1/\delta}, \frac{1}{2\lambda} \right\}
 \end{aligned}$$

where (a) follows as long as $\delta \leq 0.9$ and $\lambda \geq A$. In the case when $T_h(s) = 0$ we have $\tilde{\pi}_h(a | s) = 1/A \geq 1/\lambda$, so this lower bound still holds. Taking a union bound over arms proves the result. $\square$

Lemma 2. *As long as $\lambda \geq 4 \log(HT)$, we have*

$$\mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}}) - \mathcal{J}(\tilde{\pi})] \lesssim (1 + \alpha H) \cdot \frac{H^2 S \log T}{T} + \alpha \cdot \frac{H^2 S \lambda}{T}.$$

Proof. By the Performance-Difference Lemma we have:

$$\begin{aligned}
 \mathcal{J}(\hat{\pi}^{\text{bc}}) - \mathcal{J}(\tilde{\pi}) &= \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \left(\mathbb{E}_{a \sim \hat{\pi}_h^{\text{bc}}(s)}[Q_h^{\tilde{\pi}}(s, a)] - \mathbb{E}_{a \sim \tilde{\pi}_h(s)}[Q_h^{\tilde{\pi}}(s, a)] \right) \\
 &\leq \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \left| \mathbb{E}_{a \sim \hat{\pi}_h^{\text{bc}}(s)}[Q_h^{\tilde{\pi}}(s, a)] - \mathbb{E}_{a \sim \tilde{\pi}_h(s)}[Q_h^{\tilde{\pi}}(s, a)] \right|. \tag{4}
 \end{aligned}$$

For (s, h) with $N_h(s) > 0$, we have

$$\left| \mathbb{E}_{a \sim \hat{\pi}_h^{\text{bc}}(s)}[Q_h^{\tilde{\pi}}(s, a)] - \mathbb{E}_{a \sim \tilde{\pi}_h(s)}[Q_h^{\tilde{\pi}}(s, a)] \right| \leq \sum_{a \in \mathcal{A}} H \cdot |\hat{\pi}_h^{\text{bc}}(a | s) - \tilde{\pi}_h(a | s)|,$$

where we have used that $Q_h^{\hat{\pi}^{\text{post}}}(s, a) \in [0, H]$. Then, using the definition of $\hat{\pi}^{\text{bc}}$ and $\tilde{\pi}$ we can bound this as

$$\begin{aligned}
 &\leq \sum_{a \in \mathcal{A}} \alpha H \cdot \left| \frac{T_h(s, a)}{T_h(s)} - \frac{T_h(s, a) + \lambda/A}{T_h(s) + \lambda} \right| \\
 &= \sum_{a \in \mathcal{A}} \frac{\alpha \lambda H}{A} \cdot \left| \frac{AT_h(s, a) - T_h(s)}{T_h(s)(T_h(s) + \lambda)} \right| \\
 &\leq \sum_{a \in \mathcal{A}} \frac{\alpha \lambda H}{A} \cdot \frac{AT_h(s, a) + T_h(s)}{T_h(s)(T_h(s) + \lambda)} \\
 &= \frac{2\alpha \lambda H}{T_h(s) + \lambda}.
 \end{aligned}$$

Since $\mathbb{E}_{a \sim \hat{\pi}_h^{\text{bc}}(s)}[Q_h^{\tilde{\pi}}(s, a)] - \mathbb{E}_{a \sim \tilde{\pi}_h(s)}[Q_h^{\tilde{\pi}}(s, a)] = 0$ by construction when $T_h(s) = 0$, we then have

$$(4) \leq \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \frac{2\alpha \lambda H}{T_h(s) + \lambda}.$$

Let $\mathcal{E}$ denote the good event from Lemma 3 with $\delta = \frac{S}{T}$. Then as long as $\lambda \geq 4 \log(HT)$ we can bound the above as

$$\begin{aligned} &\leq \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \frac{2\alpha\lambda H}{T_h(s) + \lambda} \mathbb{I}\{\mathcal{E}\} + 2H^2 \cdot \mathbb{I}\{\mathcal{E}^c\} \\ &\leq \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \frac{4\alpha\lambda H}{w_h^{\pi^\beta}(s) \cdot T + \lambda} + 2H^2 \cdot \mathbb{I}\{\mathcal{E}^c\}. \end{aligned}$$

Let $\tilde{r}$ denote the reward function:

$$\tilde{r}_h(s, a) := \frac{\lambda}{w_h^{\pi^\beta}(s) \cdot T + \lambda}$$

and note that $\tilde{r} \in [0, 1]$, and

$$\sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\hat{\pi}^{\text{bc}}}(s) \cdot \frac{4\alpha\lambda H}{w_h^{\pi^\beta}(s) \cdot T + \lambda} = 4\alpha H \cdot \mathcal{J}(\hat{\pi}^{\text{bc}}; \tilde{r}).$$

By Theorem 4.4 of (Rajaraman et al., 2020), we have¹

$$\begin{aligned} \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}}; \tilde{r})] &\lesssim \mathcal{J}(\pi^\beta; \tilde{r}) + \frac{H^2 S \log T}{T} \\ &= \sum_{h=1}^H \sum_{s \in \mathcal{S}} w_h^{\pi^\beta}(s) \cdot \frac{\lambda}{w_h^{\pi^\beta}(s) \cdot T + \lambda} + \frac{H^2 S \log T}{T} \\ &\leq \frac{HS\lambda}{T} + \frac{H^2 S \log T}{T}. \end{aligned}$$

Noting that $\mathbb{E}[2H^2 \cdot \mathbb{I}\{\mathcal{E}^c\}] \leq 2H^2\delta \leq \frac{2H^2 S}{T}$ completes the proof. $\square$

Lemma 3. *With probability at least $1 - \delta$, for all (s, h) , we have*

$$T_h(s) + \lambda \geq \frac{1}{2} w_h^{\pi^\beta}(s) \cdot T + \frac{1}{2} \lambda$$

as long as $\lambda \geq 4 \log \frac{SH}{\delta}$.

Proof. Consider some (s, h) and note that $\mathbb{E}[T_h(s)/T] = w_h^{\pi^\beta}(s)$. By Bernstein's inequality, we have with probability $1 - \delta/S$:

$$T_h(s) \geq w_h^{\pi^\beta}(s) \cdot T - \sqrt{2w_h^{\pi^\beta}(s) \cdot T \cdot \log \frac{SH}{\delta}} - \frac{2}{3} \log \frac{SH}{\delta}.$$

We would then like to show that

$$\begin{aligned} &w_h^{\pi^\beta}(s) \cdot T - \sqrt{2w_h^{\pi^\beta}(s) \cdot T \cdot \log \frac{SH}{\delta}} - \frac{2}{3} \log \frac{SH}{\delta} + \lambda \geq \frac{1}{2} (w_h^{\pi^\beta}(s) \cdot T + \lambda) \\ \iff &\frac{1}{2} w_h^{\pi^\beta}(s) \cdot T + \frac{1}{2} \lambda \geq \sqrt{2w_h^{\pi^\beta}(s) \cdot T \cdot \log \frac{SH}{\delta}} + \frac{2}{3} \log \frac{SH}{\delta} \end{aligned}$$

As we have assumed $\lambda \geq 4 \log \frac{SH}{\delta}$, it suffices to show

$$\frac{1}{2} w_h^{\pi^\beta}(s) \cdot T + \log \frac{SH}{\delta} \geq \sqrt{2w_h^{\pi^\beta}(s) \cdot T \cdot \log \frac{SH}{\delta}}.$$

However, this is true by the AM-GM inequality. A union bound proves the result. $\square$

¹Note that Theorem 4.4 of (Rajaraman et al., 2020) shows an inequality in the opposite direction of what we show here: they bound $\mathcal{J}(\pi^\beta; \tilde{r}) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}}; \tilde{r})]$ instead of $\mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}}; \tilde{r})] - \mathcal{J}(\pi^\beta; \tilde{r})$. However, we see that the only place in their proof where their argument relied on this ordering is in Lemma A.8. We show in Lemma 4 that a reverse version of their Lemma A.8 holds, allowing us to instead bound $\mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}}; \tilde{r})] - \mathcal{J}(\pi^\beta; \tilde{r})$.

Lemma 4 (Reversed version of Lemma A.8 of (Rajaraman et al., 2020)). *Adopting the notation from (Rajaraman et al., 2020), we have*

$$\mathbb{E}[\Pr_{\pi^{\text{first}}}[\mathcal{E}]] \leq \frac{SH \log N}{N}$$

for $\mathcal{E}^c$ the event that within a trajectory, the policy only visits states for which $T_h(s) > 0$.

Proof. Let $\mathcal{E}_{s,h}$ denote the event that the state s is visited at step h and $T_h(s) = 0$, and $\mathcal{E}_h := \cup_{s \in \mathcal{S}} \mathcal{E}_{s,h}$. Then, by simple set inclusions, we have:

$$\mathcal{E} = \bigcup_{h \in [H]} \bigcup_{s \in \mathcal{S}} \mathcal{E}_{s,h} = \bigcup_{h \in [H]} \bigcup_{s \in \mathcal{S}} \left(\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c \right).$$

By a union bound it follows that

$$\mathbb{E}[\Pr_{\pi^{\text{first}}}[\mathcal{E}]] \leq \sum_{h \in [H]} \sum_{s \in \mathcal{S}} \mathbb{E}[\Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c]].$$

Now note that

$$\begin{aligned} \Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c] &= \Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \mid \bigcap_{h' < h} \mathcal{E}_{h'}^c] \Pr_{\pi^{\text{first}}}[\bigcap_{h' < h} \mathcal{E}_{h'}^c] \\ &= \Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \mid \bigcap_{h' < h} \mathcal{E}_{h'}^c] \Pr_{\pi^{\text{first}}}[\mathcal{E}_{h-1}^c \mid \bigcap_{h' < h-1} \mathcal{E}_{h'}^c] \Pr_{\pi^{\text{first}}}[\bigcap_{h' < h-1} \mathcal{E}_{h'}^c] \\ &\vdots \\ &= \Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \mid \bigcap_{h' < h} \mathcal{E}_{h'}^c] \cdot \prod_{h' < h} \Pr_{\pi^{\text{first}}}[\mathcal{E}_{h'}^c \mid \bigcap_{h'' < h'} \mathcal{E}_{h''}^c]. \end{aligned}$$

If the event $\bigcap_{h' < h} \mathcal{E}_{h'}^c$ holds, then up to step h no states are encountered for which $T_{h'}(s) = 0$. Thus, on such states, π^{first} and $\pi^{\text{orc-first}}$ will behave identically. It follows that $\mathbb{E}[\Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \mid \bigcap_{h' < h} \mathcal{E}_{h'}^c]] = \mathbb{E}[\Pr_{\pi^{\text{orc-first}}}[\mathcal{E}_{s,h} \mid \bigcap_{h' < h} \mathcal{E}_{h'}^c]]$. By a similar argument, we have $\Pr_{\pi^{\text{orc-first}}}[\mathcal{E}_{h'}^c \mid \bigcap_{h'' < h'} \mathcal{E}_{h''}^c] = \Pr_{\pi^{\text{first}}}[\mathcal{E}_{h'}^c \mid \bigcap_{h'' < h'} \mathcal{E}_{h''}^c]$ for each $h' < h$. Thus,

$$\Pr_{\pi^{\text{first}}}[\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c] = \Pr_{\pi^{\text{orc-first}}}[\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c].$$

It follows that

$$\mathbb{E}[\Pr_{\pi^{\text{first}}}[\mathcal{E}]] \leq \sum_{h \in [H]} \sum_{s \in \mathcal{S}} \mathbb{E}[\Pr_{\pi^{\text{orc-first}}}[\mathcal{E}_{s,h} \cap \bigcap_{h' < h} \mathcal{E}_{h'}^c]] \leq \sum_{h \in [H]} \sum_{s \in \mathcal{S}} \mathbb{E}[\Pr_{\pi^{\text{orc-first}}}[\mathcal{E}_{s,h}]].$$

From here the proof follows identically to the proof of Lemma A.8 of (Rajaraman et al., 2020). $\square$

Proof of Theorem 1. Set $\lambda = \max\{A, 4 \log(HT)\}$ and $\alpha = \frac{1}{\max\{A, H, \log(HT)\}}$. We have

$$\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}})] + \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}})] - \mathbb{E}[\mathcal{J}(\tilde{\pi})] \lesssim \frac{H^2 S \log T}{T} + (1 + \alpha H) \cdot \frac{H^2 S \log T}{T} + \alpha \cdot \frac{H^2 S \lambda}{T}$$

where we bound $\mathcal{J}(\pi^\beta) - \mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}})]$ by Theorem 4.4 of (Rajaraman et al., 2020), and $\mathbb{E}[\mathcal{J}(\hat{\pi}^{\text{bc}})] - \mathbb{E}[\mathcal{J}(\tilde{\pi})]$ by Lemma 2 since $\lambda \geq 4 \log(HT)$. By our choice of $\alpha = \frac{1}{\max\{A, H, \log(HT)\}}$, we can bound all of this as

$$\lesssim \frac{H^2 S \log T}{T}.$$

This proves the suboptimality guarantee. To show that $\tilde{\pi}$ achieves demonstrator action coverage, we apply Lemma 1 using our values of λ and α . $\square$

B.4. Optimality of Posterior Demonstrator Policy

Let $\mathcal{M}$ denote a multi-armed bandit with $A > 1$ actions where $r(a_1) = 1$ and $r(a_i) = 0$ for $i > 1$. Let $\pi^{\beta,i}$ denote the policy defined as

$$\pi^{\beta,i}(a) = \begin{cases} 1 - \alpha & a = 1 \\ \alpha & a = i \\ 0 & \text{o.w.} \end{cases}$$

for $i > 1$ and α some value we will set, and $\pi^{\beta,1}(1) = 1$. We let $\mathcal{M}^i = (\mathcal{M}, \pi^{\beta,i})$ the instance-demonstrator pair, $\mathbb{E}^i[\cdot]$ the expectation on this instance, $\mathbb{P}^i$ the distribution on this instance, and $\mathbb{P}^{i,T} = \otimes_{t=1}^T \mathbb{P}^i$.

Lemma 5. *Consider the instance constructed above. Then we have that, for $j \neq i$:*

$$\mathbb{P}^i[\hat{\pi}(i) \geq \gamma \cdot \alpha] \leq 2 \cdot \mathbb{P}^j[\hat{\pi}(i) \geq \gamma \cdot \alpha] + T \cdot \alpha.$$

Proof. This follows from Lemma A.11 of (Foster et al., 2021), which immediately gives that:

$$\mathbb{P}^i[\{\hat{\pi}(i) \geq \gamma \cdot \alpha\}] \leq 2 \cdot \mathbb{P}^j[\hat{\pi}(i) \geq \gamma \cdot \alpha] + D_H^2(\mathbb{P}^{i,T}, \mathbb{P}^{j,T}),$$

where $D_H(\cdot, \cdot)$ denotes the Hellinger distance. Since the squared Hellinger distance is subadditive we have

$$D_H^2(\mathbb{P}^{i,T}, \mathbb{P}^{j,T}) \leq T \cdot D_H^2(\mathbb{P}^i, \mathbb{P}^j).$$

By elementary calculations we see that $D_H^2(\mathbb{P}^i, \mathbb{P}^j) = \alpha$, which proves the result. $\square$

Theorem 3 (Full version of Theorem 2). *Let $\hat{\pi}$ achieve demonstrator action coverage with some parameter γ for each $\mathcal{M}^i, i \in [A]$, and some $\delta \in (0, 1/4]$, and assume that*

$$\mathcal{J}(\pi^{\beta,i}) - \mathbb{E}^i[\mathcal{J}(\hat{\pi})] \leq \xi, \quad \forall i \geq 1$$

for some $\xi > 0$. Then if $T \leq \frac{1}{4\alpha}$, it must be the case that

$$\gamma \leq \frac{\xi}{2A\alpha}.$$

In particular, setting $\xi = c \cdot \frac{\log T}{T}$ and if $\alpha = \frac{1}{2T}$, we have

$$\gamma \leq c \cdot \frac{\log T}{A}.$$

Proof. Our goal is to find the maximum value of γ such that our constraint on the optimality of $\hat{\pi}$ is met, for each $\mathcal{M}^i$. In particular, this can be upper bounded as

$$\max_{\hat{\pi}, \gamma} \gamma \quad \text{s.t.} \quad \mathbb{P}^i[\{\hat{\pi}(a) \geq \gamma \cdot \pi^{\beta,i}(a), \forall a \in \mathcal{A}\}] \geq 1 - \delta, \quad \mathcal{J}(\pi^{\beta,i}) - \mathbb{E}^i[\mathcal{J}(\hat{\pi})] \leq \xi, \quad \forall i \geq 1. \quad (5)$$

Note that for $\mathcal{M}^i, i \geq 1$, the event $\{\hat{\pi}(a) \geq \gamma \cdot \pi^{\beta,i}(a), \forall a \in \mathcal{A}\}$ is a subset of the event $\{\hat{\pi}(i) \geq \gamma \cdot \alpha\}$. This allows us to bound (5) as

$$\max_{\hat{\pi}, \gamma} \gamma \quad \text{s.t.} \quad \mathbb{P}^i[\hat{\pi}(i) \geq \gamma \cdot \alpha] \geq 1 - \delta, \quad \mathcal{J}(\pi^{\beta,i}) - \mathbb{E}^i[\mathcal{J}(\hat{\pi})] \leq \xi, \quad \forall i \geq 1. \quad (6)$$

By Lemma 5, we have that for each $i > 1$,

$$\mathbb{P}^i[\hat{\pi}(i) \geq \gamma \cdot \alpha] \leq 2 \cdot \mathbb{P}^1[\hat{\pi}(i) \geq \gamma \cdot \alpha] + T \cdot \alpha.$$

Furthermore, on $\mathcal{M}^1$ we have $\mathcal{J}(\pi^{\beta,1}) - \mathbb{E}^1[\mathcal{J}(\hat{\pi})] = \mathbb{E}^1[\sum_{i>1} \hat{\pi}(i)]$. Given this, we can upper bound (6) as

$$\max_{\hat{\pi}, \gamma} \gamma \quad \text{s.t.} \quad \mathbb{P}^1[\hat{\pi}(i) \geq \gamma \cdot \alpha] \geq \frac{1}{2} \cdot (1 - \delta - T \cdot \alpha), \quad \forall i > 1, \quad \mathbb{E}^1[\sum_{i>1} \hat{\pi}(i)] \leq \xi. \quad (7)$$

By Markov's inequality, we have

$$\mathbb{P}^1[\hat{\pi}(i) \geq \gamma \cdot \alpha] \leq \frac{\mathbb{E}^1[\hat{\pi}(i)]}{\gamma \cdot \alpha}.$$

Furthermore, since we have assumed $\delta \leq 1/4$ and $T \leq \frac{1}{4\alpha}$, we have $\frac{1}{2} \cdot (1 - \delta - T \cdot \alpha) \geq \frac{1}{4}$. We can therefore bound (7) as

$$\max_{\hat{\pi}, \gamma} \gamma \quad \text{s.t.} \quad \mathbb{E}^1[\hat{\pi}(i)] \geq \frac{1}{4} \cdot \gamma \alpha, \forall i > 1, \quad \mathbb{E}^1\left[\sum_{i>1} \hat{\pi}(i)\right] \leq \xi. \quad (8)$$

However, we see then that we immediately have

$$\gamma \leq \frac{\xi}{4(A-1)\alpha}.$$

This proves the result as long as $A > 1$. $\square$

C. Posterior Demonstrator Policy for Gaussian Demonstrator

Let $P(\cdot \mid \mu)$ denote the distribution $\mathcal{N}(\mu, \Sigma)$, where we assume μ is unknown and Σ is known. Assume that we have samples $\mathfrak{D} = \{x_1, \dots, x_T\} \sim P(\cdot \mid \mu^*)$. Let $Q_{\text{prior}} = \mathcal{N}(0, \Lambda_0)$ denote the prior on μ . Throughout this section we let $=^d$ denote equality in distribution.

Lemma 6. *Under Q_{prior} , we have that the posterior Q_{post} on μ is:*

$$Q_{\text{post}}(\cdot \mid \mathfrak{D}) = \mathcal{N}\left(\Lambda_{\text{post}} \Sigma^{-1} \cdot \sum_{t=1}^T x_t, \Lambda_{\text{post}}\right),$$

for $\Lambda_{\text{post}}^{-1} = \Lambda_0^{-1} + T \cdot \Sigma^{-1}$.

Proof. Dropping terms that do not depend on μ , we have

$$\begin{aligned} Q_{\text{post}}(\mu \mid \mathfrak{D}) &= \frac{P(\mathfrak{D} \mid \mu) Q_{\text{prior}}(\mu)}{P(\mathfrak{D})} \\ &\propto \exp\left(-\frac{1}{2} \sum_{t=1}^T (x_t - \mu)^\top \Sigma^{-1} (x_t - \mu)\right) \cdot \exp\left(-\frac{1}{2} \mu^\top \Lambda_0 \mu\right) \\ &\propto \exp\left(-\frac{1}{2} T \mu^\top \Sigma^{-1} \mu - \frac{1}{2} \mu^\top Q_{\text{prior}}^{-1} \mu + \mu^\top \Sigma^{-1} \cdot \sum_{t=1}^T x_t\right) \\ &= \exp\left(-\frac{1}{2} (\mu - \Lambda_{\text{post}} v)^\top \Lambda_{\text{post}}^{-1} (\mu - \Lambda_{\text{post}} v) + \frac{1}{2} v^\top \Lambda_{\text{post}} v\right) \end{aligned}$$

for $\Lambda_{\text{post}}^{-1} = \Lambda_0^{-1} + T \cdot \Sigma^{-1}$, and $v = \Sigma^{-1} \cdot \sum_{t=1}^T x_t$. $\square$

Lemma 7 (General version of Proposition 4). *Let*

$$\hat{\mu} = \arg \min_{\mu} \sum_{t=1}^T (\mu - \tilde{x}_t)^\top \Sigma^{-1} (\mu - \tilde{x}_t) + (\mu - \tilde{\mu})^\top \Lambda_0^{-1} (\mu - \tilde{\mu}),$$

for $\tilde{x}_t = x_t + w_t$, $w_t \sim \mathcal{N}(0, \Sigma)$, and $\tilde{\mu} \sim Q_{\text{prior}}$. Then $\hat{\mu} =^d Q_{\text{post}}(\cdot \mid \mathfrak{D})$.

Proof. By computing the gradient of the objective, setting it equal to 0, and solving for μ , we see that

$$\begin{aligned} \hat{\mu} &= (\Lambda_0^{-1} + T \Sigma^{-1})^{-1} \cdot \left(\Sigma^{-1} \cdot \sum_{t=1}^T \tilde{x}_t + \Lambda_0^{-1} \tilde{\mu} \right) \\ &= (\Lambda_0^{-1} + T \Sigma^{-1})^{-1} \cdot \Sigma^{-1} \cdot \sum_{t=1}^T x_t + (\Lambda_0^{-1} + T \Sigma^{-1})^{-1} \cdot \left(\Sigma^{-1} \cdot \sum_{t=1}^T w_t + \Lambda_0^{-1} \tilde{\mu} \right). \end{aligned}$$

Posterior Behavioral Cloning: Pretraining BC Policies for Efficient RL Finetuning

Task	Pretrained Performance			
	BC	σ -BC	DICE	POSTBC
Robomimic Lift	71.0 ± 0.5	68.0 ± 0.7	25.5 ± 4.4	69.7 ± 1.5
Robomimic Can	43.1 ± 0.9	42.3 ± 0.8	14.2 ± 2.5	44.7 ± 1.0
Robomimic Square	17.9 ± 0.7	17.8 ± 0.7	5.7 ± 0.3	18.1 ± 0.8
Libero Scene 1 (5 tasks)	23.6 ± 1.7	22.3 ± 2.1	-	25.3 ± 1.2
Libero Scene 2 (7 tasks)	11.4 ± 0.2	10.6 ± 0.8	-	12.3 ± 1.5
Libero Scene 3 (4 tasks)	39.5 ± 1.5	36.9 ± 1.8	-	40.8 ± 1.3
Libero All (16 tasks)	22.2 ± 0.3	20.9 ± 0.6	-	23.5 ± 0.5

Table 5. Comparison of success rates of all pretrained policies on Robomimic and Libero, for different pretraining approaches. Bolded text denotes best approach.

Note that the first term in the above is deterministic conditioned on $\mathcal{D}$, and the second term is mean 0 and has covariance $(\Lambda_0^{-1} + T\Sigma^{-1})^{-1}$. We see then that the mean and covariance of $\hat{\mu}$ match the mean the covariance of $Q_{\text{post}}(\cdot | \mathcal{D})$ given in Lemma 6, which proves the result. $\square$

D. Additional Experimental Details

We state a more detailed version of our procedure to compute the ensemble for POSTBC in Algorithm 2.

Algorithm 2 Posterior Variance Approximation via Ensembled Prediction

- 1: **input:** demonstration dataset $\mathcal{D}$, ensemble size K , posterior model class $\mathcal{F}$
- 2: **for** $\ell = 1, 2, \dots, K$ **do**
- 3: Set $\mathcal{D}_\ell$ to “noisy” version of $\mathcal{D}$
- 4: Fit f_ℓ by solving $f_\ell \leftarrow \arg \min_{f \in \mathcal{F}} \sum_{(s, \tilde{a}) \in \mathcal{D}_\ell} \|f_\ell(s) - \tilde{a}\|_2^2$
 $\text{cov}(\cdot) \leftarrow \sum_{\ell=1}^K (f_\ell(\cdot) - \bar{f}(\cdot))(f_\ell(\cdot) - \bar{f}(\cdot))^\top$ for $\bar{f}(\cdot) \leftarrow \frac{1}{K} \sum_{\ell=1}^K f_\ell(\cdot)$
- 5: **return** $\text{cov}(\cdot)$

Algorithm 2 fits an ensemble of predictors to a perturbed version of $\mathcal{D}$ in order to approximate a posterior sample, and uses these samples to approximate the posterior covariance. While these samples may not correspond precisely to the posterior covariance in general settings, Proposition 4 shows that in simple settings they do, suggesting that, at minimum, this is a principled approximation. In the Gaussian setting we generate a noisy $\mathcal{D}$ by perturbing the actions in $\mathcal{D}$ with Gaussian noise. In practice, however, other methods to obtain a “noisy” version of $\mathcal{D}$ can be applied as well. In particular, we found that generating $\mathcal{D}_\ell$ by *bootstrapped sampling* (Fushiki et al., 2005; Osband & Van Roy, 2015; Osband et al., 2016a)—where we sample with replacement from $\mathcal{D}$ —typically outperforms directly adding noise to the actions in $\mathcal{D}$.

Given the approximate posterior covariance $\text{cov}(\cdot)$, Section 5.1 suggests that for any s we can generate approximate samples from $\hat{\pi}^{\text{post}}(\cdot | s)$ by first sampling an action from the BC policy at s , and then perturbing the resulting action by posterior noise $w \sim \mathcal{N}(0, \text{cov}(s))$. In practice, to avoid this two-stage procedure, we can fit a single policy to $\mathcal{D}$ (which would be the BC policy) but where we perturb each action in $\mathcal{D}$ by the posterior noise. Note that the distribution this policy will fit is equivalent to the distribution produced by the above two-stage procedure, as long as we utilize an expressive generative model able to represent the posterior demonstrator policy. This is instantiated in Algorithm 1.

For all experiments we instantiate POSTBC directly as suggested in Algorithm 2 and Algorithm 1. We describe additional details on this instantiation next.

In all experiments, we parameterize f_ℓ in Algorithm 2 as an MLP (perhaps on top of a ResNet or other feature encoder, as described below). For the Robomimic experiments we let f_ℓ parameterize a Gaussian distribution and seek to model the actions in the dataset with a Gaussian, for other settings we simply have f_ℓ predict the actions directly (i.e. predicting a deterministic estimate of the actions rather than a distribution). Note that simply training f_ℓ to predict the actions directly, rather than setting f_ℓ to a generative model that seeks to model the entire action distribution, is consistent with Proposition 4—we aim to estimate a *sample* from the posterior distribution, for which it suffices to just fit a deterministic quantity, rather than fitting the entire *distribution* as generative modeling typically aims to do. Furthermore, fitting a simple predictor on the actions directly usually requires fewer training iterations than fitting, for example, a diffusion model to the entire distribution, so this also reduces the computation required to fit the ensemble.

We found in practice that using bootstrap sampling to generate the datasets $\mathcal{D}_\ell$ in Algorithm 2 performs better than

adding noise to the dataset as Proposition 4 suggests. We use both trajectory-level or state-action-level bootstrapping. For trajectory-level bootstrapping we generate $\mathcal{D}_\ell$ as in Algorithm 3. For state-action-level bootstrapping we generate $\mathcal{D}_\ell$

Algorithm 3 Trajectory-Level Bootstrap Sampling

```

1: input: demonstration dataset  $\mathcal{D}$ 
2:  $\mathcal{D}_\ell \leftarrow \emptyset$ 
3: for  $t = 1, 2, \dots$ , number of trajectories in  $\mathcal{D}$  do
4:   Sample trajectory  $\tau \sim \text{unif}(\mathcal{D})$ 
5:    $\mathcal{D}_\ell \leftarrow \mathcal{D}_\ell \cup \{\tau\}$ 
6: return  $\mathcal{D}_\ell$ 

```

as in Algorithm 4. In all experiments we parameterize our final policy with a diffusion model. Given this, Algorithm 1 is

Algorithm 4 Trajectory-Level Bootstrap Sampling

```

1: input: demonstration dataset  $\mathcal{D}$ 
2:  $\mathcal{D}_\ell \leftarrow \emptyset$ 
3: for  $t = 1, 2, \dots, |\mathcal{D}|$  do
4:   Sample state-action pair  $(s, a) \sim \text{unif}(\mathcal{D})$ 
5:    $\mathcal{D}_\ell \leftarrow \mathcal{D}_\ell \cup \{(s, a)\}$ 
6: return  $\mathcal{D}_\ell$ 

```

trained on the standard diffusion loss. Further details on each experiment are given below.

To leave room for RL improvement (i.e. to ensure performance is not saturated by the pretrained policy) we limit the number of demos per task in the pretraining dataset, for both the `Robomimic` and `Libero` experiments (see below for the precise number of trajectories used in pretraining).

For the `Robomimic` experiments, we use an MLP-based architecture, pretrain on a single-task demonstration dataset, and rely on state-based observations. For `Libero`, we utilize a diffusion transformer architecture due to (Dasari et al., 2024) and rely on image-based observations and language task conditioning. In `Libero`, we pretrain a single $\hat{\pi}^{\text{pt}}$ policy on the demonstration data from all 16 tasks (Black et al., 2024; Kim et al., 2024; Khazatsky et al., 2024), and then run RL finetuning on each individual task. On the `WidowX` experiments we utilize a U-Net architecture with image observations.

For `DSRL` and `DPPO` we utilize the publicly available implementations without modification. All policies are evaluated with 200 rollouts for `Robomimic` and 100 for `Libero`.

D.1. Robomimic Experiments

We instantiate $\hat{\pi}^\theta$ with a diffusion policy that uses an MLP architecture. For f_ℓ , we train an MLP to simply predict the action directly in $\mathcal{D}_i$ (i.e. we do not use a diffusion model for f_ℓ), but use the same architecture and dimensions for f_ℓ as the diffusion policies. We used trajectory-level bootstrapped sampling (Algorithm 3) to compute the ensemble. In all cases we pretrain on the Multi-Human `Robomimic` datasets, and in cases where we use less than the full dataset, we randomly select trajectories from the dataset to train on, using the same trajectories for each approach.

For each RL finetuning method, we sweep over the same hyperparameters for each pretrained policy method (i.e. BC, σ -BC, POSTBC), and include results for the best one. For σ -BC, we swept over values of σ and included results for the best-performing one. For all experiments results are averaged over 5 seeds (we pretrain 5 policies for each approach and run RL finetuning on each of them once, for a total of 5 RL finetuning runs per pretraining method, finetuning method, and task). For each evaluation, we roll out the policy 200 times. For `DPPO` we utilize the default hyperparameters as stated in (Ren et al., 2024), and utilize DDPM sampling. For `VALUEDICE`, we use the officially published codebase, and the default hyperparameters provided there. We found that the IQL training on the data produced by `VALUEDICE` could be somewhat unstable, and so to improve stability, for `Lift`, added LayerNorm to the IQL critic. For `DSRL`, we utilize a -1/0 success reward, and otherwise utilize a 0/1 success reward, using `Robomimic`’s built-in success detector to determine the reward. We provide hyperparameters for the individual experiments below.

Table 6. Common DSRL hyperparameters for all experiments.

Hyperparameter	Value
Learning rate	0.0003
Batch size	256
Activation	Tanh
Target entropy	0
Target update rate (τ)	0.005
Number of actor and critic layers	3
Number of critics	2
Number of environments	4

Table 7. DSRL hyperparameters for Robomimic experiments.

Hyperparameter	Lift	Can	Square
Hidden size	2048	2048	2048
Gradient steps per update	20 (POSTBC, BC), 10 (σ -BC)	20 (POSTBC, BC), 10 (σ -BC)	20 (POSTBC, BC), 10 (σ -BC)
Noise critic update steps	20	10	10
Discount factor	0.99	0.99	0.999
Action magnitude	1.5	1.5	1.5
Initial steps	24000	24000	32000

Table 8. Hyperparameters for pretrained policies for Robomimic DSRL experiments.

Hyperparameter	Lift	Can	Square
Dataset size (number trajectories)	5	10	30
Action chunk size	4	4	4
train denoising steps	100	100	100
inference denoising steps	8	8	8
Hidden size	512	1024	1024
Hidden layers	3	3	3
Training epochs	3000	3000	3000
Ensemble size (POSTBC)	100	100	100
Ensemble training epochs (POSTBC)	10000	6000	3000
Posterior noise weight α (POSTBC)	1	0.5	1
Uniform noise σ (σ -BC)	0.1	0.1	0.05

Table 9. Best-of- N hyperparameters for Robomimic experiments.

Hyperparameter	Lift	Can	Square
Total gradient steps	2000000	2000000	2000000
IQL τ (1000 rollouts)	0.5 (BC, POSTBC), 0.7 (σ -BC), 0.9 (DICE)	0.7	0.7
IQL τ (2000 rollouts)	0.5 (BC), 0.7 (σ -BC, POSTBC), 0.9 (DICE)	0.7	0.7
Discount factor	0.999	0.999	0.999

 Table 10. Hyperparameters for pretrained policies for Robomimic Best-of- N experiments.

Hyperparameter	Lift	Can	Square
Dataset size (number trajectories)	20	300	300
Action chunk size	1	1	1
Train denoising steps	100	100	100
Hidden size	512	1024	1024
Hidden layers	3	3	3
Training epochs	3000	3000	3000
Ensemble size (POSTBC)	100 (1000 rollouts), 10 (2000 rollouts)	10	10
Ensemble training epochs (POSTBC)	3000	500	500
Posterior noise weight α (POSTBC)	1 (1000 rollouts), 2 (2000 rollouts)	1	1 (1000 rollouts), 2 (2000 rollouts)
Uniform noise σ (σ -BC)	0.1	0.025	0.025

Table 11. Hyperparameters for pretrained policies for Robomimic DPPO experiments.

Hyperparameter	Lift	Can	Square
Dataset size (number trajectories)	5	10	30
Action chunk size	4	4	4
train denoising steps	100	100	100
Hidden size	512	1024	1024
Hidden layers	3	3	3
Training epochs	3000	3000	3000
Ensemble size (POSTBC)	100	100	10
Ensemble training epochs (POSTBC)	3000	6000	3000
Posterior noise weight α (POSTBC)	0.5	0.25	1
Uniform noise σ (σ -BC)	0.1	0.05	0.05

D.2. Libero Experiments

For Libero, we utilize the transformer architecture from (Dasari et al., 2024) for $\hat{\pi}^\theta$. For POSTBC we use state-action bootstrap sampling (Algorithm 3) to generate $\mathcal{D}_\ell$. For f_ℓ , we utilize the same ResNet and tokenizer as $\hat{\pi}^\theta$, but simply utilize a 3-layer MLP head on top of it—trained to predict the actions directly—rather than a full diffusion transformer. For the Best-of- N experiments, POSTBC utilizes a diagonal posterior covariance estimate (that is, instead of computing the full covariance matrix as prescribed by Algorithm 2, we compute the covariance dimension-wise, and construct a diagonal covariance matrix from this), while for the DSRL runs it is trained with the full matrix posterior covariance estimate. We

train on Libero 90 data from the first 3 scenes of Libero 90—KITCHEN SCENE 1, KITCHEN SCENE 2, and KITCHEN SCENE 3—and use 25 trajectories from each task in each scene. For task conditioning, we condition $\hat{\pi}^\theta$ on the BERT language embedding (Devlin et al., 2019) of the corresponding text given for that task in the Libero dataset.

For each RL finetuning method, we sweep over the same hyperparameters for each pretrained policy method (i.e. BC, σ -BC, POSTBC), and include results for the best one. We utilize the DSRL-SAC variant of DSRL from (Wagenmaker et al., 2025a). For σ -BC, we swept over values of σ and included results for the best-performing one. The DSRL experiments are averaged over 3 different pretraining runs per method, and one DSRL run per pretrained run. The Best-of- N experiments are averaged over 2 different pretraining runs per method. For each evaluation, we roll out the policy 100 times. In all cases, we utilize a -1/0 success reward, using Libero’s built-in success detector to determine the reward.

We provide hyperparameters for the individual experiments below.

Table 12. DSRL hyperparameters for all Libero experiments.

Hyperparameter	Value
Learning rate	0.0003
Batch size	256
Activation	Tanh
Target entropy	0
Target update rate (τ)	0.005
Number of actor and critic layers	3
Layer size	1024
Number of critics	2
Number of environments	1
Gradient steps per update	20
Discount factor	0.99
Action magnitude	1.5
Initial episode rollouts	20

Table 13. Best-of- N hyperparameters for all Libero experiments.

Hyperparameter	Value
IQL learning rate	0.0003
IQL batch size	256
IQL β	3
Activation	Tanh
Target update rate	0.005
Q and V number of layers	2
Q and V layer size	256
Number of critics	2
N (Best-of- N samples)	32
IQL gradient steps	50000
IQL τ	0.9
Discount factor	0.99

Table 14. Hyperparameters for DiT diffusion policy in Libero experiments.

Hyperparameter	Value
Batch size	150
Learning rate	0.0003
Training steps	50000
LR scheduler	cosine
Warmup steps	2000
Action chunk size	4
Train denoising steps	100
Inference denoising steps	8
Image encoder	ResNet-18
Hidden size	256
Number of Heads	8
Number of Layers	4
Feedforward dimension	512
Token dimension	256
Ensemble size (POSTBC)	5
Ensemble training steps (POSTBC)	25000
Ensemble layer size	512
Ensemble number of layers	3
Posterior noise weight (POSTBC)	2 (DSRL run), 4 (Best-of- N run)
Uniform noise σ (σ -BC)	0.05

D.3. WidowX Experiments



Figure 8. Setup for WidowX “Put corn in pot” task.



Figure 9. Setup for WidowX “Pick up banana” task.

For the WidowX experiment, we collect 10 demonstrations on the “Put corn in pot” task. For the diffusion policy, we utilize a U-Net architecture with ResNet image encoder. For the POSTBC ensemble predictors, we utilize a ResNet image encoder with MLP regression head, trained to directly predict the action in the dataset. For both BC and POSTBC, we pretrain the policy on the 10 demonstrations, then roll out the pretrained policy 100 times on each task, manually resetting the scene each time and classifying each trajectory as success or failure. We utilize a 0/1 reward (every step is given a reward of 0 unless it succeeds, when it is given a reward of 1). We then train an IQL Q -function on the rollout data and, at test time, roll out the pretrained policy, sampling N actions at each step, and choosing the action with the maximum Q -value. For IQL, we utilize an MLP-based architecture, and to process the images, we utilize image features from a ResNet encoder pretrained on the Bridge v2 dataset (Walke et al., 2023). For both BC and POSTBC, we try different values of N and different number of IQL training steps, and report the results for the best-performing values for each approach. All hyperparameters for the diffusion policy are given in Table 15, and for IQL in Table 16.

Table 15. Hyperparameters for pretrained policies for WidowX experiments.

Hyperparameter	Both WidowX tasks
Action chunk size	1
Train denoising steps	100
Inference denoising steps	16
Image encoder	ResNet-18
U-Net channel size	[256, 512, 1024]
U-Net kernel size	5
Training epochs	800
Ensemble predictor hidden size	512
Ensemble predictor hidden layers	3
Ensemble size (POSTBC)	10
Ensemble training epochs (POSTBC)	300
Posterior noise weight α (POSTBC)	1

 Table 16. Best-of- N hyperparameters for WidowX experiments.

Hyperparameter	Put corn in pot	Pick up banana
IQL learning rate	0.0003	0.0003
IQL batch size	256	256
IQL β	3	3
Activation	Tanh	Tanh
Target update rate	0.005	0.005
Q and V number of layers	2	2
Q and V layer size	256	256
Number of critics	2	2
N (Best-of- N samples)	4	16
IQL gradient steps	400000 (BC), 700000 (POSTBC)	100000
IQL τ	0.7	0.7
Discount factor	0.97	0.97

D.4. Additional Ablations

For all ablation experiments, other than the hyperparameter we vary, we utilize the hyperparameters given in Section D.1. In Figure 12 we provide an additional ablation on the dataset size for Robomimic Square, and in Figure 13 provide additional qualitative results on Libero.

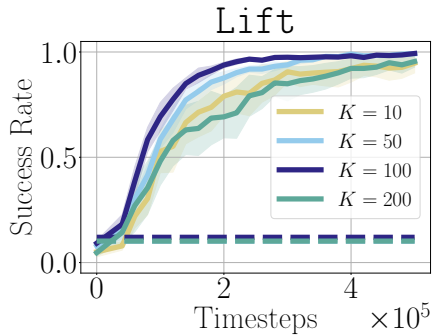


Figure 10. Sensitivity of POSTBC with DSRL finetuning to ensemble size. Dashed lines denote pretrained policy performance.

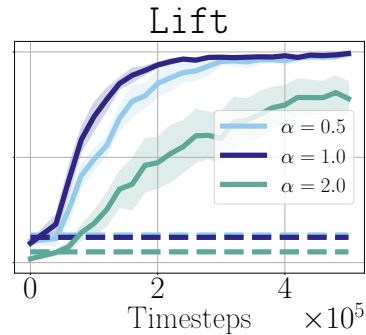


Figure 11. Sensitivity of POSTBC with DSRL finetuning to posterior weight. Dashed lines denote pretrained policy performance.

We consider the sensitivity of POSTBC to two key hyperparameters: the size of the ensemble (K) and the weight of the posterior variance (α in Algorithm 1). We illustrate the performance of POSTBC on Robomimic Lift varying these parameters in Figures 10 and 11. In Figure 10 we see that POSTBC performs best with a moderately sized ensemble ($K = 100$), but is not particularly sensitive to ensemble size as long as it is not too small or too large. In Figure 11 we see that setting α too large can hurt the performance of POSTBC, but that otherwise the performance of POSTBC is relatively stable with respect to α . We note as well that setting α too large not only hurts the performance of the RL finetuning, but also causes the pretrained performance to drop. This is to be expected—even with the carefully tuned noise POSTBC adds to the policy in pretraining, if the weight of this noise is too large the perturbations it induces will cause performance to drop below that of the BC policy. In general, throughout our experiments, we found that $\alpha = 1$ typically performs well.

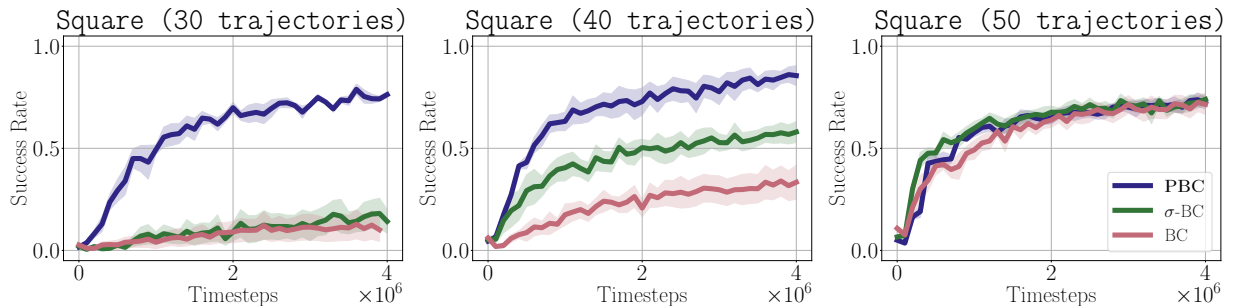


Figure 12. Comparison of DSRL finetuning performance combined with different BC pretraining approaches on Robomimic Square, varying the number of trajectories in the dataset the policies are pretrained on. As can be seen, the finetuning performance of policies pretrained with POSTBC is largely unaffected by the size of the pretraining dataset, while BC and σ -BC are both very sensitive to dataset size. For large enough datasets (50 trajectories), BC and σ -BC perform as well as POSTBC. This is to be expected—if we train on enough data, our uncertainty will be low, so POSTBC will essentially reduce to BC. These results illustrate that POSTBC gracefully interpolates between settings where BC overfits to small amounts of data, hurting its finetuning performance, and settings where BC is sufficient for effective finetuning.

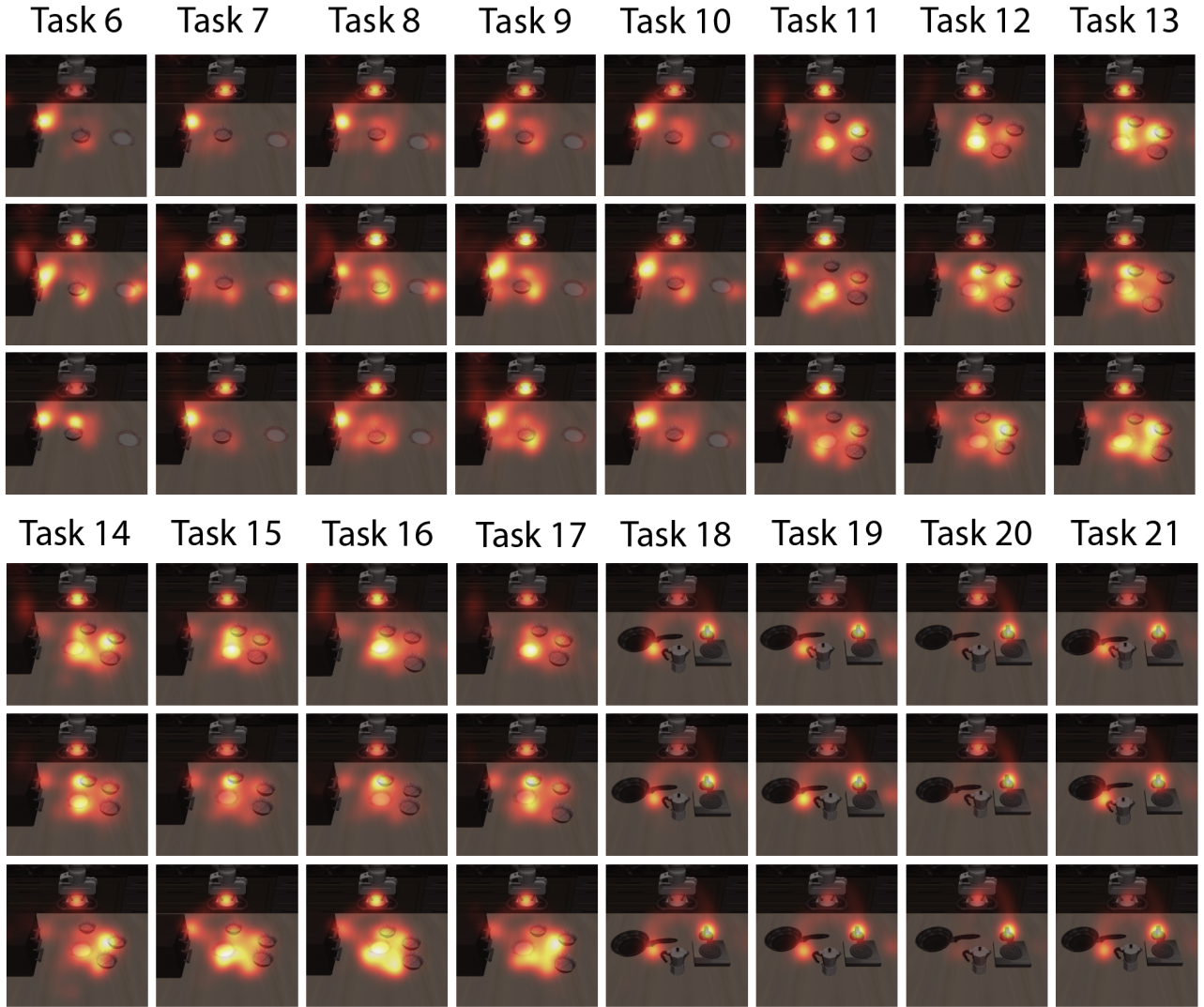


Figure 13. Additional density heatmaps of pretrained policies on tasks 6-21 from Libero 90. See Table 17 for task commands.

Table 17. Task descriptions for Libero tasks in Kitchen Scene 1–3.

Task ID	Task description
Task 6	Open the bottom drawer of the cabinet
Task 7	Open the top drawer of the cabinet
Task 8	Open the top drawer of the cabinet and put the bowl in it
Task 9	Put the black bowl on the plate
Task 10	Put the black bowl on top of the cabinet
Task 11	Open the top drawer of the cabinet
Task 12	Put the black bowl at the back on the plate
Task 13	Put the black bowl at the front on the plate
Task 14	Put the middle black bowl on the plate
Task 15	Put the middle black bowl on top of the cabinet
Task 16	Stack the black bowl at the front on the black bowl in the middle
Task 17	Stack the middle black bowl on the back black bowl
Task 18	Put the frying pan on the stove
Task 19	Put the moka pot on the stove
Task 20	Turn on the stove
Task 21	Turn on the stove and put the frying pan on it
